

“Linux System Information and Resource Monitoring Tool”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

VENKATA SAI KRISHNA

2520030027

PUJITH PAVAN KUMAR

2520030029

AJAI MANIKANTA

2520030320

BANDLA VINAY

2520030437

Under the Esteemed Guidance of

Dr. Muniraju Naidu V

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled **“Linux System Information and Resource Monitoring Tool”** is a record of Bonafide work of **VENKATA SAI KRISHNA - 2520030027, PUJITH PAVAN KUMAR - 2520030029, AJAI MANIKANTA - 2520030320, BANDLA VINAY - 2520030437** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VENKATA SAI KRISHNA	2520030027
PUJITH PAVAN KUMAR	2520030029
AJAI MANIKANTA	2520030320
BANDLA VINAY	2520030437

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **“Linux System Information and Resource Monitoring Tool”** is a bonafide work done and submitted by **VENKATA SAI KRISHNA – 2520030027, PUJITH PAVAN KUMAR – 2520030029, AJAI MANIKANTA – 2520030320, BANDLA VINAY – 2520030437** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance and support of many people. We take this opportunity to express our sincere thanks to everyone who helped us, directly or indirectly, during the course of this work.

We are greatly thankful to our Course Coordinator Dr. K. Anusha, and our project guide

Dr Muniraju Naidu Department of Computer Science and Engineering, for their constant encouragement, valuable suggestions and guidance throughout the Operating Systems and Systems Programming course project.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the Department of Computer Science and Engineering, for providing the facilities and the academic environment required to complete this project.

We thank all the teaching and non-teaching staff members of the department who supported us during this work. Finally, we sincerely thank our parents, friends and classmates for their help and cooperation.

VENKATA SAI KRISHNA	2520030027
PUJITH PAVAN KUMAR	2520030029
AJAI MANIKANTA	2520030320
BANDLA VINAY	2520030437

Abstract

Modern operating systems manage several hardware resources, such as the processor, main memory, storage devices and network interfaces, while many processes execute at the same time. Understanding how these resources are being used is important for identifying performance problems, but on Linux this information is normally obtained by running separate commands, each with its own output format. For students learning Operating Systems, this makes it harder to see how the individual resource-management concepts fit together.

This project presents the Linux System Information and Resource Monitoring Tool, a menu-driven terminal application developed in C for Linux/Ubuntu. The tool collects information directly from the Linux `/proc` filesystem and from interfaces such as `uname()`, `sysconf()`, `getpwuid()` and `statvfs()`, and presents it through a single interface. Its modules display system and processor details; CPU utilisation calculated from two samples of `/proc/stat`; RAM and swap usage from `/proc/meminfo`; mounted file systems and their space usage; running processes with PID, parent PID and state, together with a process hierarchy tree; system uptime; a combined system health summary; network interface counters with a live bandwidth view; the top processes by CPU and memory; process search; load averages and context-switch counters; and loaded kernel modules. A process manager can send signals to a process and change its priority. Resource alerts compare CPU, memory and root-disk usage with warning and critical thresholds, and a logging facility appends timestamped snapshots to the file `monitor_log.txt`.

The project demonstrates operating system services and system calls, process management, memory management, file system management and resource management. Inter-process communication is not directly implemented in the current version. The tool is a lightweight educational application and is not intended to replace professional monitoring software. It was compiled with GCC and executed on Ubuntu running under WSL, and its terminal output is documented in this report.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Declaration		i
	Certificate		ii
	Acknowledgement		iii
	Abstract		iv
1	Introduction		1–3
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	3
2	OS Concepts and Background Study		4–9
		2.1 Operating System Concepts Related to the Project	4
		2.2 Existing Methods / Approaches	6
		2.3 Proposed Approach	7
		2.4 Advantages of the Proposed Approach	8
		2.5 CO-wise Mapping	8
3	System Requirements		10–11
		3.1 Hardware Requirements	10
		3.2 Software Requirements	10
		3.3 Operating System Environment — Ubuntu / WSL	10
		3.4 Development Tools and Compilers	11
4	Technology and Tools Used		12–15
		4.1 C Programming	12
		4.2 GCC Compiler	12
		4.3 Linux System Calls / APIs	13
		4.4 Linux /proc Filesystem	13
		4.5 Linux Commands and Utilities	14
		4.6 Other Tools	15
5	System Architecture and Design		16–20
		5.1 Overall System Architecture	16

S. No.	Chapter	Topics / Subtopics	Page No.
		5.2 System Workflow	16
		5.3 Flowchart / Data Flow	17
		5.4 Module Description	18
		5.5 Input and Output Design	19
6	Implementation		21–41
		6.1 Module-wise Implementation	21
		6.2 Algorithm / Pseudocode	33
		6.3 Core C Implementation	35
		6.4 Linux System Calls / APIs Used	38
		6.5 Process / Thread Implementation	39
		6.6 IPC / Synchronization Implementation	40
		6.7 Memory Management Implementation	40
		6.8 User Interface Implementation	41
7	Execution and Demonstration		42–52
		7.1 Compilation in Ubuntu	42
		7.2 Execution Commands	42
		7.3 Main Menu	43
		7.4 System Information	44
		7.5 CPU Monitoring	44
		7.6 Memory Monitoring	45
		7.7 Disk Monitoring	45
		7.8 Process Monitoring	46
		7.9 System Uptime	46
		7.10 System Health	47
		7.11 Network Monitoring	47
		7.12 Top Resource Processes	47
		7.13 Process Search	48
		7.14 Resource Alerts	48
		7.15 Resource Logging	48
		7.16 Interactive Process Manager	49
		7.17 System Load Averages and Context Switches	50
		7.18 Kernel Modules Explorer	50
		7.19 Testing	50
		7.20 Demonstration of OS Concepts	52

S. No.	Chapter	Topics / Subtopics	Page No.
8	Conclusion		53–55
		8.1 Summary of the Project	53
		8.2 Objectives Achieved	53
		8.3 Results Achieved	54
		8.4 Skills and Learning Outcomes	54
		8.5 Limitations	54
		8.6 Future Scope	55
9	References		56–57
		9.1 Books	56
		9.2 Research Papers	56
		9.3 Linux / GNU Documentation	56
		9.4 Online Resources	57

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

An operating system is the system software that sits between the hardware of a computer and the programs that run on it. One of its main duties is resource management: the processor has to be shared between many processes, main memory has to be allocated and reclaimed, files and storage devices have to be organised, and network interfaces have to be made available to applications. Because all of these resources are limited, the operating system continuously keeps account of how they are being used.

When a system is slow or unresponsive, the cause is usually found by looking at this accounting information: a process may be using most of the CPU time, the free memory may be low and the system may have started using swap space, or a file system may be nearly full. System monitoring is therefore an important part of administering any computer, and it is also a good way for students to see the concepts of an operating systems course at work on a real machine.

Linux is well suited for this purpose because it exposes a large amount of kernel information to user programs. The `/proc` pseudo-file system presents data such as CPU counters, memory statistics, uptime, network counters and per-process details as ordinary text files, which can be opened and read with normal file operations. In addition, Linux and POSIX provide system calls and library functions such as `uname()`, `sysconf()` and `statvfs()` that return system and file system information directly. A C program can use these interfaces without any special libraries, which makes Linux a convenient platform for learning system programming.

1.2 Problem Statement

Linux provides many commands for inspecting a running system, but each command shows only one kind of information and has its own output format. A user who wants to check CPU load, memory, disk space, running processes and network traffic has to run several different commands and combine the results mentally. For a beginner, it is also not obvious where these numbers come from or how they are calculated, so the link between the output on the screen and the operating system concepts behind it is easily lost.

For students of Operating Systems there is therefore a need for a simple tool that brings the commonly needed information into one place, and whose source code is small and readable enough to be studied. This project addresses that need. It does not attempt to replace the existing Linux

utilities; its purpose is to provide a unified, menu-driven, educational interface that reads the same kernel interfaces directly and shows how the values are obtained.

1.3 Scope of the Project

The project is a menu-driven terminal application written in C for Linux/Ubuntu. Its scope covers the following areas:

- Display of system information: distribution, kernel, architecture, host name, processor model, number of logical cores, frequency and current user.
- CPU utilisation measured from two samples of `/proc/stat`, with an optional live view.
- RAM and swap usage, and file-system space usage for mounted partitions.
- Listing of running processes with PID, parent PID and state, a process hierarchy tree, and process counts by state.
- System uptime, a system health summary, load averages and scheduler counters.
- Network interface byte counters and a live bandwidth view.
- Top processes by CPU and memory, process search, and an interactive process manager (signals and priority).
- A list of loaded kernel modules with search.
- Resource threshold alerts and resource usage logging to a text file.

The following limitations define the boundaries of the project. The application runs only on Linux, because it depends on `/proc` and Linux-specific interfaces. It has a text interface only; there is no graphical dashboard, no remote monitoring and no database. The log file is written only when the user selects the logging option. The current version does not implement inter-process communication mechanisms such as pipes, shared memory, message queues or semaphores, and it does not display inode statistics. Values reported under WSL may differ from those of a native Linux installation (Section 3.3).

1.4 Objectives of the Project

The objectives of the project are:

1. To develop a Linux-based system monitoring application using the C programming language.
2. To collect system information using Linux system interfaces such as the `/proc` filesystem, `uname()`, `sysconf()` and `statvfs()`.
3. To monitor CPU utilisation from the kernel CPU time counters.

4. To monitor RAM and swap memory usage.
5. To monitor disk space usage of the mounted file systems.
6. To inspect running processes, their states and the process hierarchy.
7. To monitor network interface activity and live bandwidth.
8. To identify the processes that consume the most CPU and memory.
9. To search for processes by name or PID, and to send signals and change priority through a process manager.
10. To provide resource threshold alerts and to record resource usage in a log file.
11. To understand and demonstrate practical Operating System concepts through a working program.

1.5 Importance of the Application

For Operating Systems students, the application shows how abstract concepts appear in a running system. The process list shows process identifiers, parent relationships and states; the CPU module shows how utilisation is derived from the kernel counters; the memory and disk modules show how the operating system accounts for RAM, swap and file-system blocks. Because every value is read from a `/proc` file or a system call that is visible in the source code, students can trace each number on the screen back to its origin.

For Linux learners and beginner system administrators, the tool gives a quick overview of the state of a machine without having to remember several commands. The health summary and the alert module give an immediate indication of whether CPU, memory or the root file system is under pressure. The process manager shows how signals such as SIGTERM, SIGKILL, SIGSTOP and SIGCONT and the nice value are used to control processes.

The project also gives the team practical experience in system programming: parsing kernel-provided text files, handling terminal input in raw mode, working with fixed-size buffers and dynamic memory, and presenting the results in a readable form.

1.6 Target Users / Audience

- **Students** who are studying Operating Systems and Systems Programming and want a working reference implementation.
- **Linux learners** who want to understand what the `/proc` filesystem contains.
- **Operating Systems learners** who want to relate process, memory, file-system and resource management concepts to real output.

- **Beginner system administrators** who need a simple overview of CPU, memory, disk, process and network state.
- **Users interested in basic system monitoring** on Ubuntu or other Linux distributions, including Ubuntu running under WSL.

CHAPTER 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

This section describes the operating system concepts on which the project is based. For each concept, the last sentence explains how it appears in the project.

2.1.1 Operating System Services

An operating system offers services to users and programs: program execution, file-system access, input and output, resource allocation, accounting and protection. A user program does not access hardware or kernel data directly; it requests these services through the kernel. In this project, the application uses these services to obtain information about the machine, for example the kernel name, release and architecture, the number of online processors and the identity of the current user.

2.1.2 System Calls

A system call is the controlled entry point through which a user program requests a service from the kernel. In C on Linux, many system calls are used through library wrapper functions. The project uses `uname()` to obtain system identification data, `sysconf()` to obtain the number of online processors and the page size, `statvfs()` to obtain file-system statistics, `kill()` to send signals, `setpriority()` to change a scheduling priority, and file-related calls through the C library (`fopen()`, `fgets()`, `fclose()`) and directory functions (`opendir()`, `readdir()`) to read `/proc`.

2.1.3 Process Management

A process is a program in execution. The kernel creates processes, schedules them on the CPU, and keeps information about each of them, such as its identifier (PID), the identifier of its parent (PPID), its state and its resource usage. The project does not create new processes; it observes the existing ones by reading the directory entries and status files under `/proc`, and it can send signals to them or change their priority through the process manager.

2.1.4 Process States

During its life a process moves between states. Linux reports the state of a process as a single letter in `/proc/[PID]/stat`. The states that matter for this project are listed in Table 2.1.

Table 2.1: Process states reported by Linux and their handling in the project

State	Meaning	Handling in the project
R	Running or runnable	Counted as Running; shown in green in the process list
S	Interruptible sleep (waiting for an event)	Counted as Sleeping

State	Meaning	Handling in the project
D	Uninterruptible sleep (usually waiting for I/O)	Counted as Sleeping
T	Stopped (for example by SIGSTOP)	Counted as Stopped
Z	Zombie (terminated, not yet reaped by its parent)	Counted as Zombie; shown in red in the process list
Other	Other kernel-defined states	Included in the total count only

2.1.5 Process Hierarchy

Every process except the first has a parent. On a Linux system the first user-space process (PID 1, for example `systemd`) is the ancestor of the others, so processes form a tree. The PPID field of `/proc/[PID]/stat` gives the link from a child to its parent. The project reads PID and PPID of every process and builds a tree by repeatedly searching for the children of each process, starting from the processes whose parent is 0.

2.1.6 CPU Utilization

The Linux kernel keeps cumulative counters of the time the processors have spent in different modes, such as user mode, system mode and idle. The first line of `/proc/stat` gives these counters summed over all processors. CPU utilisation cannot be read as a single value; it is calculated from the change of these counters during an interval. The project reads the counters twice, 300 ms apart, and calculates the fraction of the interval that was not idle (Section 6.1.2).

2.1.7 Memory Management

The operating system divides physical memory into fixed-size pages and allocates them to processes, the kernel and caches. Linux reports the total memory, the memory that is available for new applications without swapping, and several breakdown values such as buffers, cached memory and shared memory in `/proc/meminfo`. For individual processes, `/proc/[PID]/statm` reports the size in pages and the resident set size, that is, the number of pages currently held in RAM. The project uses these files for the memory module and for the top-memory process list.

2.1.8 Virtual Memory and Swap

Virtual memory allows the system to use more memory than the installed RAM by moving less-used pages to a swap area on disk. The total and free swap space are reported in `/proc/meminfo` as `SwapTotal` and `SwapFree`. The memory module of the project calculates the swap in use from these two values, and it reports when no swap space is configured.

2.1.9 File System Management

The file system organises data on storage devices into files and directories and keeps track of which blocks are used and which are free. Each mounted file system has a device, a mount point and a type. The project reads the list of mounted file systems from `/proc/mounts` and then asks the kernel for the block statistics of each mount point with `statvfs()`.

2.1.10 Disk Space

Disk space is described by the block size, the total number of blocks, and the number of free blocks. Multiplying these by the fragment size gives the capacity and free space in bytes. The project converts these values into gigabytes and calculates the percentage of space in use for each mounted file system.

2.1.11 Inodes

In Unix-like file systems an inode stores the metadata of a file, and the number of inodes is limited when the file system is created. A file system can therefore become full either because its blocks are used or because its inodes are exhausted. The `statvfs()` structure contains inode counts, but the current version of the project does not read or display them; only block-based space usage is reported. Inode statistics are listed as future scope in Section 8.6.

2.1.12 Resource Management

Resource management is the task of allocating limited resources (CPU time, memory, storage and network bandwidth) among competing processes and of keeping account of their use. Monitoring is the observation side of resource management. The project monitors CPU, memory, disk, processes and network, compares usage with warning and critical thresholds, and records snapshots in a log file.

2.1.13 Linux /proc Filesystem

The `/proc` file system is a pseudo-file system: it does not store data on disk, but the kernel generates its contents when a file is read. It contains system-wide files such as `/proc/stat`, `/proc/meminfo`, `/proc/uptime` and `/proc/net/dev`, and one numeric directory for every running process, such as `/proc/525/stat`. Because these are text files, they can be read with ordinary file functions. `/proc` is the main data source of the project (Section 4.4).

2.2 Existing Methods / Approaches

Linux already provides well-established tools for monitoring. The most common ones are described below.

- **top** shows a continuously updated list of processes together with a summary of CPU, memory and load average.
- **htop** is an interactive process viewer with a coloured display and scrolling; it is often installed separately.
- **ps** prints a snapshot of the current processes with selectable columns such as PID, PPID and state.
- **free** reports the amount of used and free RAM and swap.
- **df** reports the space used and available on mounted file systems.
- **uptime** reports how long the system has been running and the load averages.
- **Direct inspection of /proc**, for example with the cat command, gives the raw kernel data on which many of these utilities are based.

Utilities such as top, ps, free and uptime obtain most of their data from /proc, while df uses file-system calls similar to statvfs(). These tools are mature, complete and suitable for professional use. The present project does not replace them. Its purpose is to combine selected monitoring functions in one small C program, so that the way the data is obtained and processed is visible in the source code. Table 2.2 compares the existing tools with the project.

Table 2.2: Feature comparison of existing Linux tools and the proposed tool

Feature	Existing Linux tool / interface	In this project	Remark
CPU utilisation	top, /proc/stat	Yes	Overall CPU only, 300 ms sample
RAM and swap	free, /proc/meminfo	Yes	Total, used, available, cache, buffers, shared, swap
Disk space	df	Yes	Block-based space usage
Inode usage	df -i	No	Not implemented in this version
Process list	ps, top, htop	Yes	First 30 entries shown, plus counts
Process tree	ps, htop (tree view)	Yes	Depth limited to 4 levels
Uptime	uptime, /proc/uptime	Yes	Uptime and cumulative idle time
Load average	uptime, top	Yes	With context-switch counters
Network counters	/proc/net/dev	Yes	Bytes received/sent, live KB/s
Top processes	top, htop	Yes	Top 10 by CPU and by RSS

Feature	Existing Linux tool / interface	In this project	Remark
Kill / renice	kill, renice	Yes	SIGTERM, SIGKILL, SIGSTOP, SIGCONT, nice value
Kernel modules	lsmod, /proc/modules	Yes	View and search only
All in one menu	Separate commands	Yes	Single menu-driven program

2.3 Proposed Approach

The proposed system is a C program for Linux/Ubuntu with a menu-driven terminal interface. When the user selects a feature, the program reads the required information directly from the Linux interfaces, that is, from /proc files and from system calls and library functions. The collected values are then processed: counters are converted into percentages, bytes are converted into larger units, and processes are sorted or filtered. The results are displayed in the terminal using headers, tables, progress bars and colour-coded status labels. The program can compare CPU, memory and disk usage with thresholds, and it can write a timestamped snapshot of the readings to a log file.

The program is a single source file, main.c, that is compiled with GCC into the executable monitor. No external libraries, databases or services are required.

2.4 Advantages of the Proposed Approach

- **Simple interface:** all features are reached from one numbered menu.
- **Centralised monitoring:** system information, resources, processes, network and kernel data are available in one program.
- **Lightweight:** the program is a single executable with no background service.
- **Written in C:** the code works at the level of system calls and file reads, close to the operating system.
- **Direct interaction with Linux interfaces:** data is read from /proc and system calls, not from other utilities.
- **Educational value:** each displayed value can be traced to a specific file or function.
- **Demonstrates OS concepts:** processes, memory, file systems, system calls and resource management are all visible in the program.
- **No external database:** the only persistent data is the text file monitor_log.txt.
- **Easy to compile and run:** one gcc command builds the program.

2.5 CO-wise Mapping

Table 2.3 maps the project to the Course Outcomes (CO1 to CO6) of the Operating Systems and Systems Programming course. The mapping lists only features that are present in the source code. CO3 (Inter-Process Communication) is not directly implemented in the current version and is marked accordingly.

Table 2.3: Mapping of the project to the Course Outcomes

CO	Course Outcome area	Implementation in the project	Status
CO1	Operating System services / system calls	uname(), sysconf(), getuid(), getpwuid(); /etc/os-release and /proc-based system information (System Information, CPU, uptime and other modules)	Implemented
CO2	Process management	PID, PPID and state from /proc/[PID]/stat; process counts by state; process hierarchy tree; process search; per-process CPU and memory analysis; process control with kill() and setpriority()	Implemented
CO3	Inter-Process Communication	Pipes, shared memory, message queues and semaphores are not used in the current version. (Signal handling and kill() are used only for terminal clean-up and process control; see Section 6.6.)	Not directly implemented
CO4	Memory management	RAM usage, available memory, cached, buffer and shared memory, swap usage from /proc/meminfo; per-process resident memory from /proc/[PID]/statm. (Active and Inactive values are read but not displayed.)	Implemented
CO5	File system management	Mounted file systems from /proc/mounts; total, used and free space with statvfs(); file reading and writing for /proc files and the log file. (Inode counts are not displayed.)	Implemented (space usage)
CO6	Resource management	CPU, memory, disk, process and network monitoring; load averages; resource thresholds and alerts; resource usage logging	Implemented

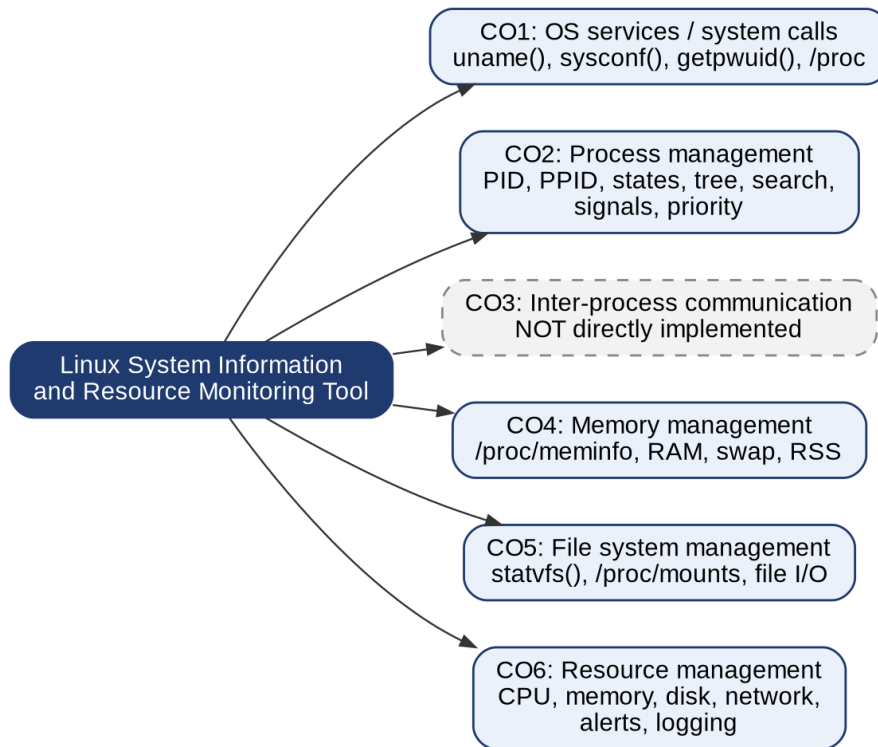


Figure 2.1: Mapping of the project to Course Outcomes CO1–CO6

Figure 2.1 shows the same mapping as a diagram. CO3 is drawn with a dashed outline to show that it is not directly implemented.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The program is small and does not need special hardware. Table 3.1 lists the minimum requirements for compiling and running it.

Table 3.1: Hardware requirements

Component	Requirement
Processor	Any modern x86/x64 processor
RAM	Minimum 4 GB
Storage	At least 1 GB of free space for the project and tools
Input device	Keyboard
Output device	Display

The project was developed and executed on a laptop with a 13th Gen Intel Core i5-1335U processor reporting 12 logical cores, as shown in the System Information screenshot (Figure 7.2). This is not a requirement of the program.

3.2 Software Requirements

Table 3.2: Software requirements

Software	Purpose
Ubuntu Linux / Linux environment	Operating environment that provides /proc and the Linux system interfaces
GCC compiler	Compiles main.c into the executable monitor
Visual Studio Code	Editing the source code
Linux terminal	Compiling, running and testing the program
Standard C libraries	Input/output, string handling, memory allocation, sorting and time functions
POSIX / Linux APIs	uname(), sysconf(), statvfs(), directory functions, termios, select(), signals and process control

3.3 Operating System Environment — Ubuntu / WSL

The project runs on Linux. It can be used directly on a native Ubuntu installation, or on Ubuntu running under the Windows Subsystem for Linux (WSL). WSL is therefore a convenient option for students who use Windows, but it is not mandatory. The screenshots in this report were taken on Ubuntu under WSL; the kernel string visible in Figure 7.2 (microsoft-standard-WSL2) confirms this.

The `/proc` file system is available in both cases, because both run a Linux kernel. Some of the values can nevertheless differ between native Linux and WSL. For example, the processes visible in WSL are only those of the Linux environment, the memory and CPU figures are those given to the WSL virtual machine, and information such as kernel modules may be restricted or different. The program cannot be compiled and run natively on Windows or macOS because it depends on `/proc` and on Linux-specific behaviour.

3.4 Development Tools and Compilers

GCC (GNU Compiler Collection) is used to compile the program. **Visual Studio Code** is used as the source-code editor, and the **Linux terminal** is used to run the compiler and the executable. The source code is kept in a **Git/GitHub** repository (V-max-OSSP/V-Max_OSSP_Project) created for the course.

CHAPTER 4

TECHNOLOGY AND TOOLS USED

This chapter describes the technologies, system interfaces and tools that are actually used in `main.c`.

Table 4.1: Technologies used

Technology	Use in the project
C programming language	Implementation of the whole application (single file <code>main.c</code>)
GCC	Compilation into the executable monitor
Linux / Ubuntu (also under WSL)	Operating environment that provides <code>/proc</code> and the Linux interfaces
<code>/proc</code> filesystem	Main source of CPU, memory, process, network and kernel data
Linux / POSIX APIs	System calls and library functions for system information, files, signals and terminal control
ANSI escape sequences	Colours, screen clearing and progress bars in the terminal
Visual Studio Code	Source-code editor
Git / GitHub	Source-code repository

4.1 C Programming

C is the language of Unix and Linux, and the Linux system-call interface and the C library are written for it. It is suitable for this project for the following reasons:

- **System programming:** C gives direct access to system calls, file descriptors, signals and terminal control.
- **Interaction with the operating system:** the data structures used by the kernel interfaces (for example `struct utsname`, `struct statvfs`, `struct termios`) are defined as C structures.
- **Low-level resource access:** the program works with fixed-size buffers, explicit parsing of text read from `/proc`, and explicit memory allocation with `malloc()` and `free()`.
- **Linux APIs:** POSIX and Linux functions are documented for C in the Linux man-pages.
- **Small, fast executables:** the compiled program has no runtime or virtual machine and starts immediately.

4.2 GCC Compiler

GCC translates the C source code into an executable file. The compilation steps (preprocessing, compilation, assembly and linking) are performed by one command:

```
| gcc -Wall -Wextra main.c -o monitor
```

The option `-Wall` enables a large set of common warnings and `-Wextra` enables additional ones, which helps to find mistakes such as unused variables or suspicious comparisons. The option `-o monitor` names the output executable. The source file defines `_GNU_SOURCE` at the top, which makes GNU extensions such as `strcasestr()` available. After successful compilation, the program is started with `./monitor`.

4.3 Linux System Calls / APIs

The following functions are used in `main.c`. The complete list with the module in which each is used is given in Table 6.6 of Chapter 6.

- **uname():** fills a struct `utsname` with the kernel name, release, machine architecture and host name.
- **sysconf():** returns the number of online processors (`_SC_NPROCESSORS_ONLN`) and the page size (`_SC_PAGESIZE`).
- **getuid() and getpwuid():** identify the current user and return the user name and home directory.
- **statvfs():** returns file-system statistics such as block size, total blocks and free blocks for a mount point.
- **opendir(), readdir() and closedir():** scan the `/proc` directory to find the numeric process directories.
- **fopen(), fgets(), fscanf(), fprintf() and fclose():** read the `/proc` files and write the log file.
- **sscanf():** extracts numbers and words from lines of text read from `/proc`.
- **qsort():** sorts the process list by CPU usage and by memory usage.
- **strcasestr():** case-insensitive substring search used for process search and kernel-module search.
- **kill() and setpriority():** send signals to a process and change its nice value in the process manager.
- **access():** checks that the directory `/proc/[PID]` exists before the process manager acts on a PID.
- **select(), read(), tcgetattr() and tcsetattr():** read key presses without blocking and switch the terminal to raw mode for the live monitors.
- **signal() and atexit():** restore the terminal when the program is interrupted or exits.
- **time(), localtime() and strftime():** create the timestamp written to the log file.
- **malloc() and free():** allocate and release the process arrays.

4.4 Linux /proc Filesystem

The /proc file system is the main data source of the application. Table 4.2 lists the files that the program reads and the information taken from each of them.

Table 4.2: /proc files and other Linux interfaces used by the project

File	Information provided to the application	Module
/proc/cpuinfo	Processor model name and CPU frequency (MHz)	System Information
/proc/stat	Aggregate CPU time counters (first line); context switches, forks, running and blocked process counts	CPU, Health, Alerts, Logging, Top Processes, Load
/proc/meminfo	MemTotal, MemAvailable, Cached, Buffers, Shmem, Active, Inactive, SwapTotal, SwapFree	Memory, Health, Alerts, Logging
/proc/uptime	Seconds since boot and cumulative idle time	System Uptime
/proc/net/dev	Received and transmitted byte counters per network interface	Network Monitoring
/proc/[PID]/stat	Process name, state, parent PID, user and system CPU time	Processes, Top Processes, Search
/proc/[PID]/statm	Process memory size in pages; the resident set size is used	Top Processes
/proc/mounts	Device, mount point and type of mounted file systems	Disk Usage
/proc/loadavg	1, 5 and 15 minute load averages, runnable/total scheduling entities, last PID	Load Averages
/proc/modules	Loaded kernel modules with size, use count and state	Kernel Modules
/etc/os-release	Distribution name (PRETTY_NAME); this file is not under /proc	System Information

The numeric directory names under /proc are the PIDs of the running processes. The program identifies them by testing whether the first character of a directory entry is a digit.

4.5 Linux Commands and Utilities

Only a few shell commands are needed to build and run the project:

Table 4.3: Commands used to build and run the project

Command	Use
cd	Change to the project directory, for example <code>cd ~/linux-system-monitor</code>
ls	List the files in the directory (main.c, monitor, monitor_log.txt)
<code>gcc -Wall -Wextra main.c -o monitor</code>	Compile the source file into the executable monitor
<code>./monitor</code>	Run the program
<code>cat monitor_log.txt</code>	Display the log file written by the logging module (optional)
sudo	Needed only if the process manager is used to set a negative nice value or to signal processes owned by other users

4.6 Other Tools

Visual Studio Code was used for writing and editing the code. **Git and GitHub** were used to keep the project repository. No other programming languages, frameworks, databases or external libraries are used in the project.

CHAPTER 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a simple layered design (Figure 5.1). The user interacts with a menu-driven interface implemented in `main()`. Each menu choice calls one monitoring module, which is a C function. A module reads the data it needs from the Linux system interfaces: `/proc` files, system calls and file-system functions. The raw values are then processed, for example by taking the difference of two counters or by converting bytes into gigabytes, and the result is shown in the terminal. The resource logging module additionally writes a record to the file `monitor_log.txt`.

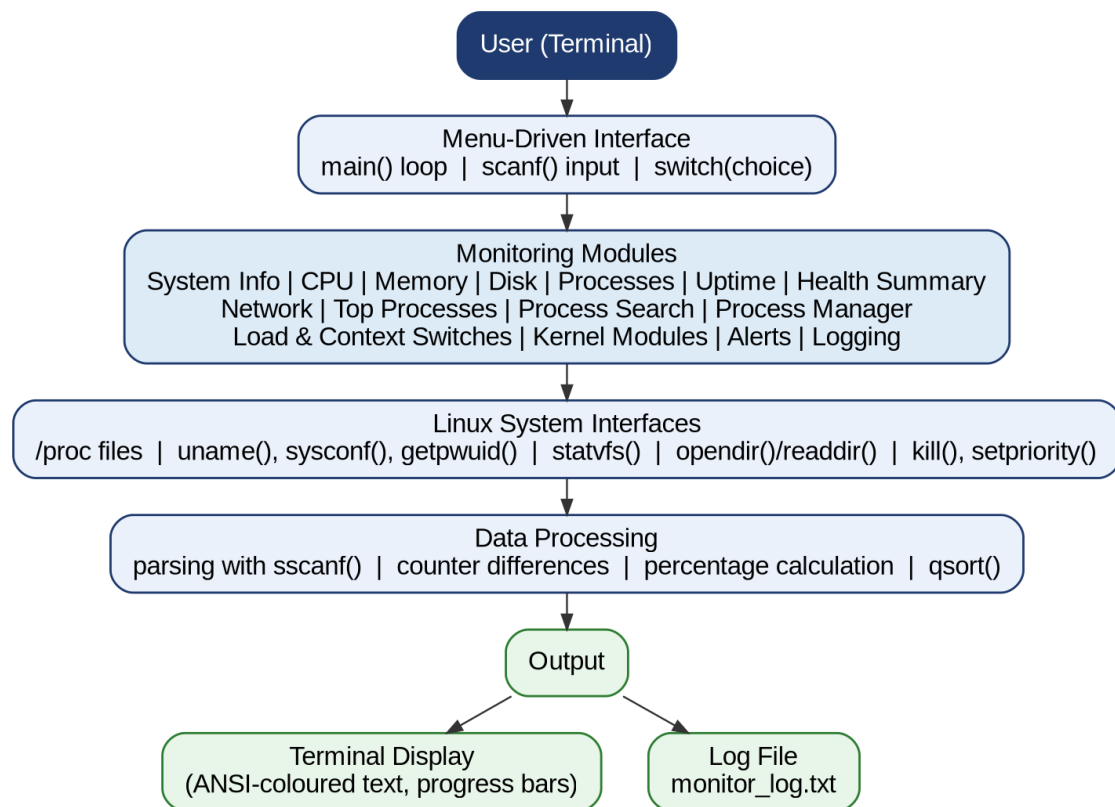


Figure 5.1: Overall system architecture

The monitoring modules are: System Information, CPU Monitoring, Memory Monitoring, Disk Monitoring, Process Monitoring, Uptime, Health Summary, Network Monitoring, Top Processes, Process Search, Process Manager, Load and Context Switches, Kernel Modules Explorer, Alerts and Logging.

5.2 System Workflow

The program works in the following steps (Figure 5.2):

1. The application starts and installs the signal handler.
2. The main menu is displayed.
3. The user selects a feature by entering its number.
4. The program accesses the Linux interface that the feature needs.
5. The data is collected.
6. The data is processed or calculated.
7. The result is displayed.
8. The user returns to the main menu or exits.

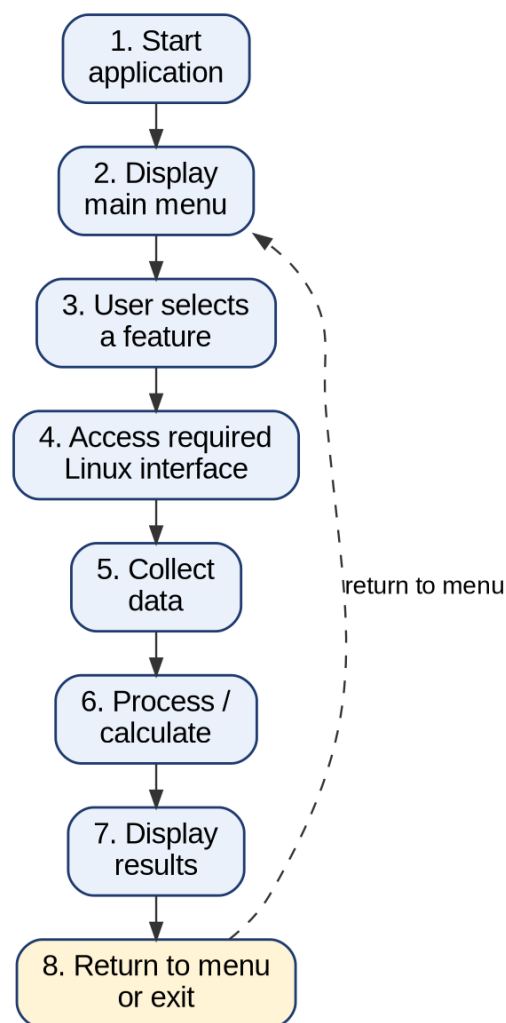


Figure 5.2: System workflow

5.3 Flowchart / Data Flow

Figure 5.3 shows the flowchart of the main program. After a feature finishes, the function `afterFeature()` asks whether to return to the main menu or to exit. Option 16 of the main menu also

ends the program. Invalid input is detected, an error message is printed and the menu is displayed again.

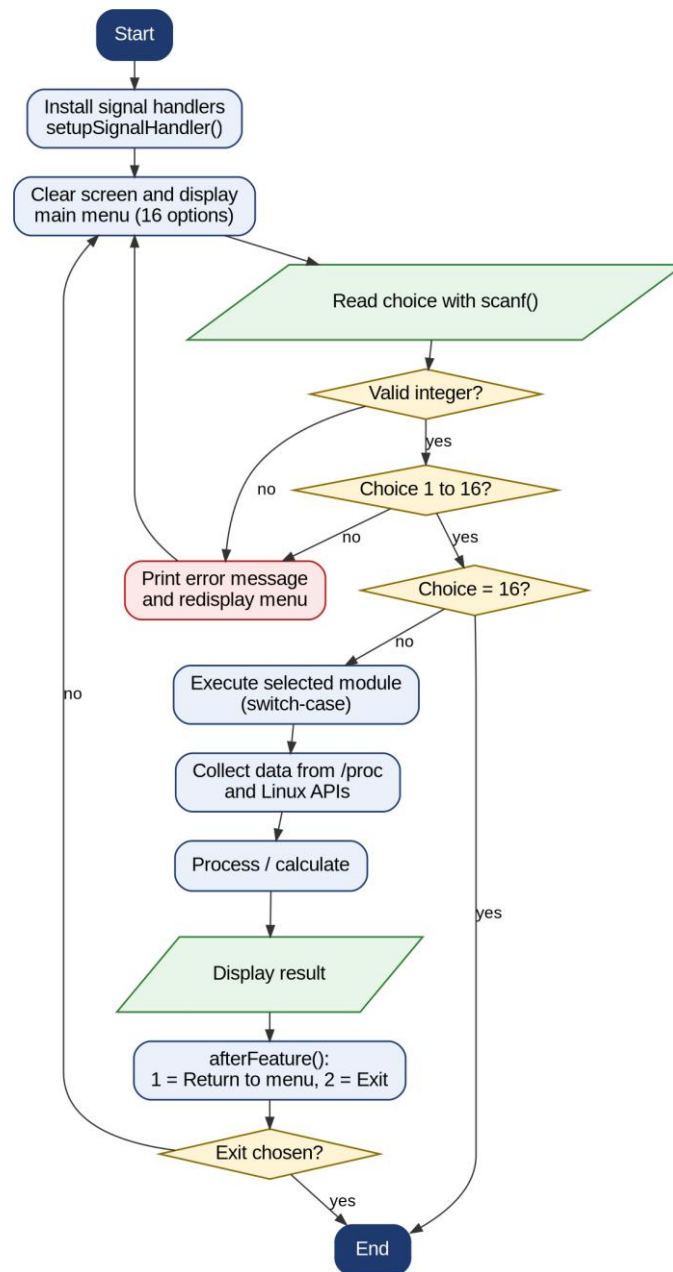


Figure 5.3: Main program flowchart

5.4 Module Description

Table 5.1 describes each module of the application, the Linux interfaces it uses and what it displays.

Table 5.1: Module description

No.	Module	Purpose	Linux interface / API used	Output
1	System Information	Show OS, kernel, processor and user details	uname(), sysconf(), getpwuid(), /etc/os-release, /proc/cpuinfo	Distribution, kernel, architecture, host name, CPU model, cores, frequency, user
2	CPU Usage	Measure CPU utilisation; optional live view	/proc/stat, usleep(), select(), termios	Progress bar, percentage, status badge
3	Memory Usage	Show RAM and swap usage	/proc/meminfo	RAM and swap bars, total/used/available, cached, buffers, shared
4	Disk Usage	Show space usage of mounted file systems	/proc/mounts, statvfs()	Table with device, mount, type, total, free, usage and bar
5	Process Monitoring	List processes, count states, show hierarchy	opendir(), readdir(), /proc/[PID]/stat	Counts, process table, process tree
6	System Uptime	Show time since boot	/proc/uptime	Days/hours/minutes/seconds, total hours, cumulative idle time
7	System Health Summary	Combined view of CPU, RAM, disk and processes	Functions of modules 2, 3, 4, 5	Gauges, process counts, composite status
8	Network Monitoring	Show interface byte counters and live speed	/proc/net/dev	RX/TX table; live KB/s per interface
9	Top Resource Processes	Find top 10 processes by CPU and by memory	/proc/[PID]/stat, /proc/[PID]/statm, sysconf(), qsort()	Two ranked tables
10	Process Search	Find processes by name or PID	opendir(), readdir(), /proc/[PID]/stat, strcasestr()	Matching processes with PID, PPID, state
11	Process Manager	Send signals or change priority	kill(), setpriority(), access()	Action menu and result message
12	Load and Context Switches	Show load averages and scheduler counters	/proc/loadavg, /proc/stat, sysconf()	Load gauges, run queue, context switches
13	Kernel Modules Explorer	List loaded kernel modules and search them	/proc/modules, strcasestr()	Module table (first 20) and search results
14	Resource Alerts	Compare CPU, RAM and disk with thresholds	getCPUUsage(), getMemoryUsage(), getDiskUsage()	Status badges and alert/notice messages
15	Resource Logging	Record a snapshot in a file and show recent entries	time(), localtime(), strftime(), fopen(), fprintf()	Confirmation, last 10 log entries

The main menu groups these modules into three sections: System and Hardware Metrics (options 1–7), Process and Network Management (options 8–11) and Kernel Diagnostics and Telemetry (options 12–15). Option 16 is Exit.

5.5 Input and Output Design

Input. All input is given from the keyboard. The user mainly enters a menu number. Some modules need additional input: a process name or PID for the search, a PID and an action for the process manager, a nice value, a module-name substring for the kernel-module search, and a choice in the submenus. In the live CPU and network monitors the keys q, ESC or End stop the

monitoring. Menu input is read with `scanf()` and checked: a value that is not an integer produces an error message, and a number outside 1–16 produces an “Invalid choice” message.

Output. Output is shown in the terminal. Each screen has a coloured header, sections, tables and, where useful, progress bars whose colour changes from green to yellow at 70 % and to red at 85 %. Status badges (NORMAL, WARNING, CRITICAL) summarise the level of a resource. The logging module also produces file output: each snapshot is appended to `monitor_log.txt` in the current directory. The screens are shown in Chapter 7.

Table 5.2: Module input and output

Module	Input from user	Output
System Information	None	Information screen
CPU Usage	Option 0 (live monitor) or 1 (return); q / ESC / End to stop	CPU bar; live CPU line
Memory / Disk / Uptime / Health	None	Gauges and tables
Process Monitoring	Submenu: 1 tree, 2 process manager, 3 return	Process table; tree
Network Monitoring	Option 0 (live bandwidth) or 1 (return)	Interface table; live speed table
Top Resource Processes	None	Top 10 by CPU, top 10 by memory
Process Search	Process name (substring) or PID	Matching process list
Process Manager	PID; action 1–6; nice value for action 5	Result message
Load and Context Switches	None	Load averages and counters
Kernel Modules Explorer	Option 1 (search) with module-name substring, or 2	Module table; matches
Resource Alerts	None	Status and alert messages
Resource Logging	Option 1 (view last 10 entries) or 2	Confirmation; <code>monitor_log.txt</code>

CHAPTER 6

IMPLEMENTATION

This chapter describes the implementation of the project as it exists in `main.c`. The program is a single C source file of about 1,240 lines. It defines four data structures, a set of helper functions for terminal handling and formatting, one function for each monitoring module, and a `main()` function that implements the menu loop. Only features that are present in the source code are described.

6.1 Module-wise Implementation

6.1.1 System Information

The function `systemInformation()` collects static information about the machine and the current user.

- **Kernel and architecture:** `uname()` fills a struct `utsname`. The program prints the operating system name and release (`info.sysname` and `info.release`), the machine architecture (`info.machine`) and the host name (`info.nodename`).
- **Distribution:** the file `/etc/os-release` is read line by line and the value of the `PRETTY_NAME` field is used. If the file cannot be read, the default text “Linux” is shown.
- **Processor:** `/proc/cpuinfo` is read line by line. A line starting with “model name” (or “Processor”) gives the CPU model, and a line starting with “cpu MHz” gives the frequency. Since the file contains one block per logical processor, the values of the last block that was read are kept. Defaults are “Generic CPU” and “Unknown”.
- **Logical cores:** `sysconf(_SC_NPROCESSORS_ONLN)` returns the number of processors that are online.
- **Current user:** `getpwuid(getuid())` returns the user name and home directory. If it fails, the environment variables `USER` and `HOME` are used. The numeric user ID is printed with `getuid()`.

6.1.2 CPU Usage

CPU utilisation is calculated by the function `getCPUUsage()`, which is also used by the health summary, the alert module and the logging module. The first line of `/proc/stat` has the form `cpu user nice system idle iowait irq softirq steal ...` and contains cumulative times, in clock ticks, summed over all logical processors. The function reads the first eight values, waits 300 ms with `usleep(300000)`, and reads them again.

The calculation is:

```
Total CPU Time = user + nice + system + idle + iowait + irq + softirq + steal
Idle Time      = idle + iowait
Busy Time      = Total CPU Time - Idle Time

CPU Usage (%)  = (Busy Time Difference / Total Time Difference) x 100
```

The differences are taken between the second and the first sample. If the total difference is zero, the result is 0 %. The sampling interval of this function is 300 ms, so the value represents the average load over that short interval. The result is shown as a progress bar with a status badge.

The CPU module (`cpuUsage()`) offers an additional live mode. When the user selects option 0, the terminal is switched to raw mode (Section 6.8) and the program repeatedly waits up to 400 ms for a key press, calls `getCPUUsage()` (300 ms of sampling) and rewrites a single line on the screen using a carriage return. The display is therefore refreshed roughly every 0.7 seconds. The live mode stops when the user presses `q`, `Q`, `x`, `X`, `ESC` or the End key. It is a sampled, near-real-time display and not a millisecond-level monitor. Only overall CPU usage is shown; usage per logical core is not calculated.

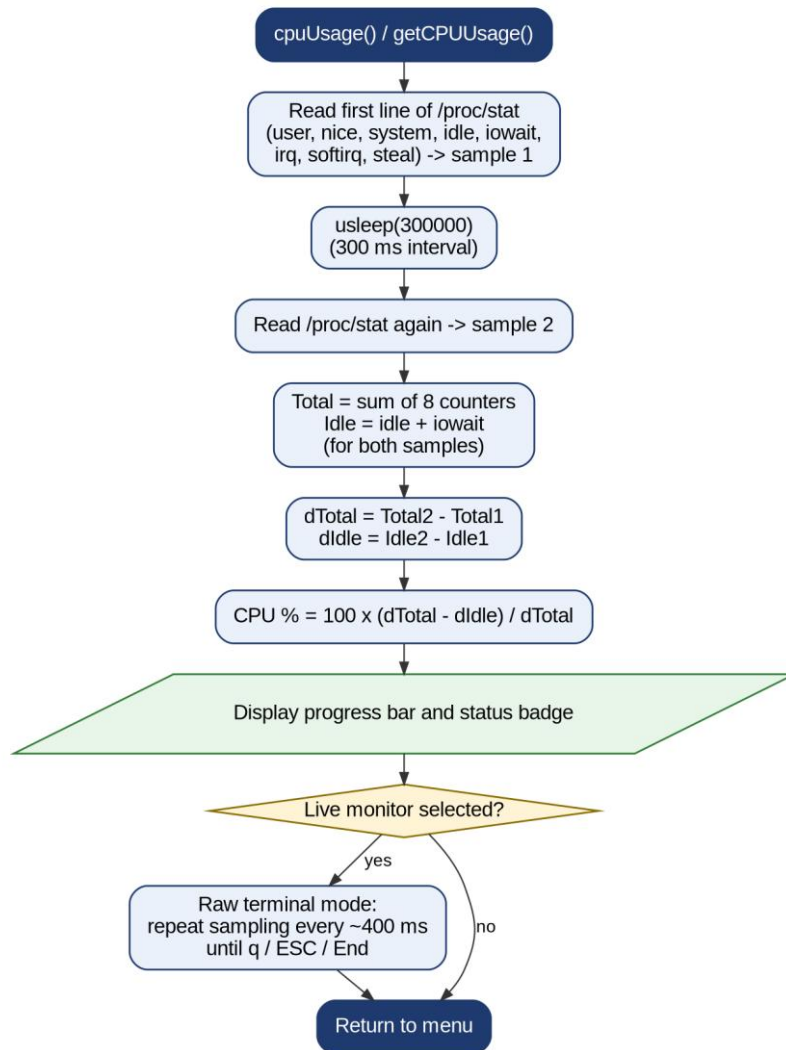


Figure 6.1: CPU monitoring flow

6.1.3 Memory Usage

The function `memoryUsage()` reads `/proc/meminfo` line by line and extracts the following fields (all in kilobytes): `MemTotal`, `MemAvailable`, `Cached`, `Buffers`, `Shmem`, `Active`, `Inactive`, `SwapTotal` and `SwapFree`. The values are calculated as follows:

```

Used Memory    = MemTotal - MemAvailable
RAM Usage (%)  = Used Memory / MemTotal x 100
Swap Used      = SwapTotal - SwapFree
Swap Usage (%) = Swap Used / SwapTotal x 100    (only if SwapTotal > 0)
  
```

The program displays the total, used and available memory in gigabytes (kilobytes divided by 1024×1024), and a memory breakdown with the `Cached`, `Buffers` and `Shared (Shmem)` values. If `SwapTotal` is zero, the message “No active swap space configured” is printed; otherwise a swap bar and the total and used swap are shown. The values `Active` and `Inactive` are read from the file but are not displayed in the current version.

The memory status is shown with a status badge. The same thresholds are used throughout the program (Table 6.1). A lighter function, getMemoryUsage(), returns only the RAM percentage and is used by the health summary, alert and logging modules.

Table 6.1: Status levels used by the progress bars and status badges

Usage	Badge	Progress bar colour
Below 70 %	NORMAL	Green
70 % to below 85 %	WARNING	Yellow
85 % and above	CRITICAL	Red

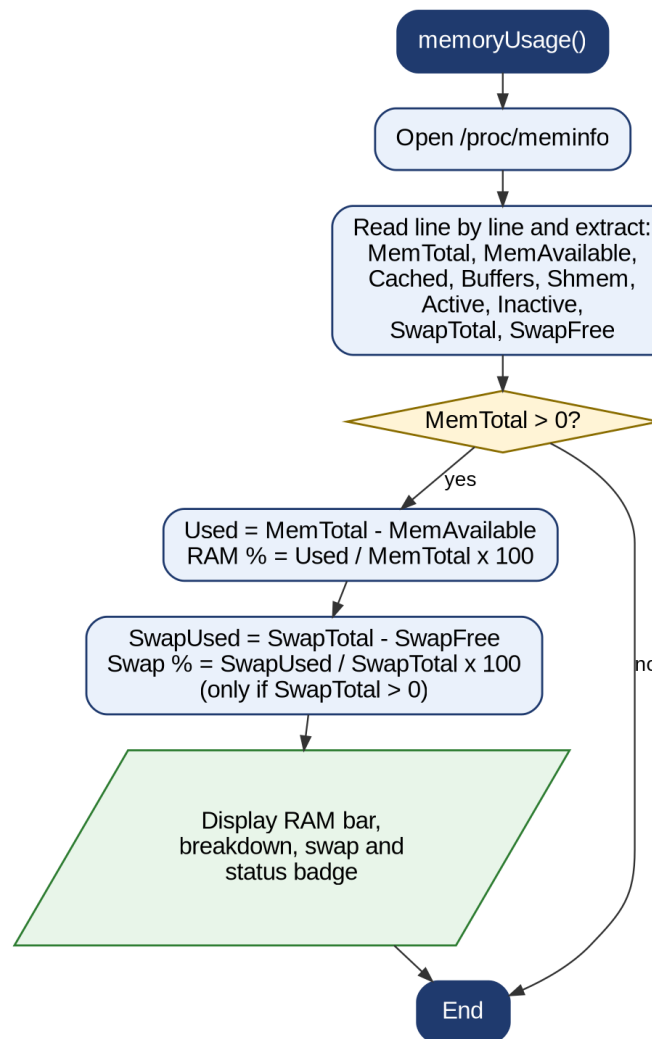


Figure 6.2: Memory monitoring flow

6.1.4 Disk Usage

The function `diskUsage()` first reads `/proc/mounts`. For every line it extracts the device, the mount point and the file-system type. A mount is considered if its device name begins with `/dev/` or if its mount point is the root directory. For each such mount point, `statvfs()` is called, and mounts with

zero blocks or with a mount point that was already listed are skipped. At most 32 mounts are stored in an array of DiskMount structures. If no mount could be read, the function falls back to `statvfs("/")` and labels the entry “/dev/root”, “/” and “ext4”.

The helper `setDiskMount()` converts the `statvfs()` fields into gigabytes:

```
Total Space = f_blocks x f_frsize
Free Space  = f_bavail x f_frsize
Used Space  = Total Space - Free Space
Usage (%)   = Used Space / Total Space x 100
```

For every mount the program prints the device, mount point, file-system type, total size, free space and usage percentage, followed by a progress bar. For the root file system alone, the function `getDiskUsage()` is used by the health summary, alert and logging modules; it calculates the used space as the total minus $f_bfree \times f_frsize$. The `statvfs()` structure also contains inode counts, but these are not used in the current version, so inode usage is not reported.

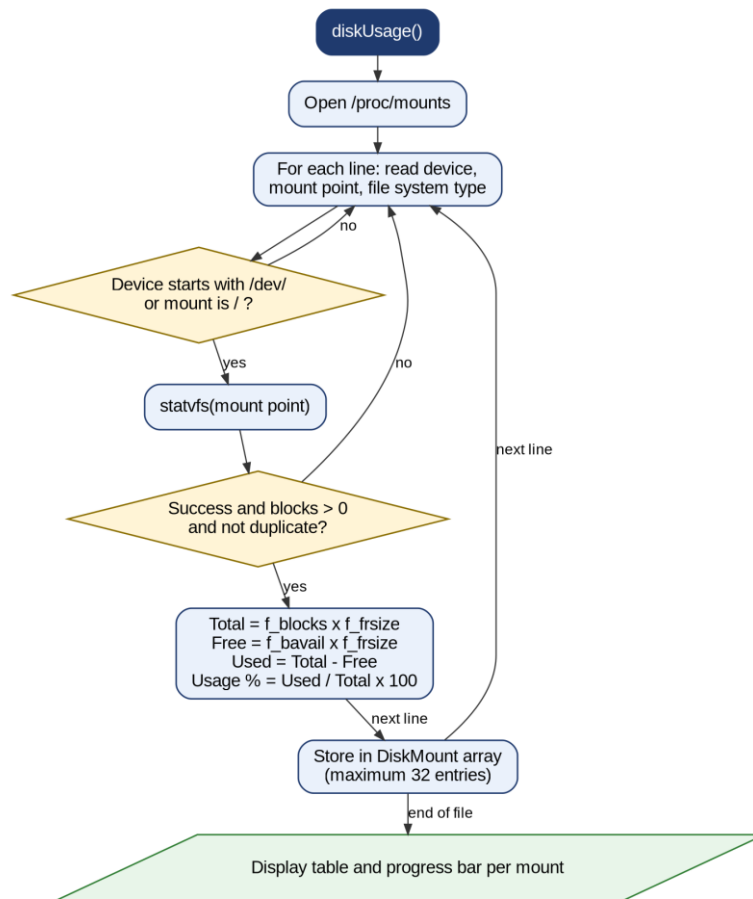


Figure 6.3: Disk monitoring flow

6.1.5 Process Monitoring

Process monitoring is implemented in `processMonitoring()`. The function allocates an array of `ProcessInfo` structures (capacity `MAX_PROCESSES = 4096`) with `malloc()`, opens the directory

/proc with opendir() and reads it with readdir(). Directory entries whose name starts with a digit are process directories. For each of them the helper readProcStat() reads /proc/[PID]/stat.

The file /proc/[PID]/stat is a single line in which the second field is the command name enclosed in parentheses. Because the name can itself contain spaces or parentheses, readProcStat() finds the first '(' and the last ')' and copies the text between them as the name. The remaining fields are parsed with sscanf() starting two characters after the closing parenthesis: the state (a single character) and the parent PID. When the CPU time is requested (Top Processes), the function continues to the 14th and 15th fields, utime and stime, and returns their sum.

The ProcessInfo structure stores pid, ppid, state and name. While reading, the function counts the processes by state: R is counted as running, S and D as sleeping, T as stopped and Z as zombie. Processes in other states are included only in the total. The screen shows these counts, followed by a table with PID, PPID, STATE and COMMAND for the first 30 processes in the order of the /proc directory, and a note with the number of remaining processes.

A submenu then allows the user to view the process tree, to open the process manager, or to return. The tree is produced by the recursive function printTreeRecursive(). It prints every process whose parent PID equals the given value, indents it according to its level, and then calls itself with that process as the new parent. The recursion starts with parent PID 0 and is limited to a depth of 4 levels. Because the function scans the whole array for each parent, its running time grows roughly with the square of the number of processes, which is acceptable for the small process counts of a typical system. A further function, getProcessCounts(), returns only the total and running counts for the health summary.

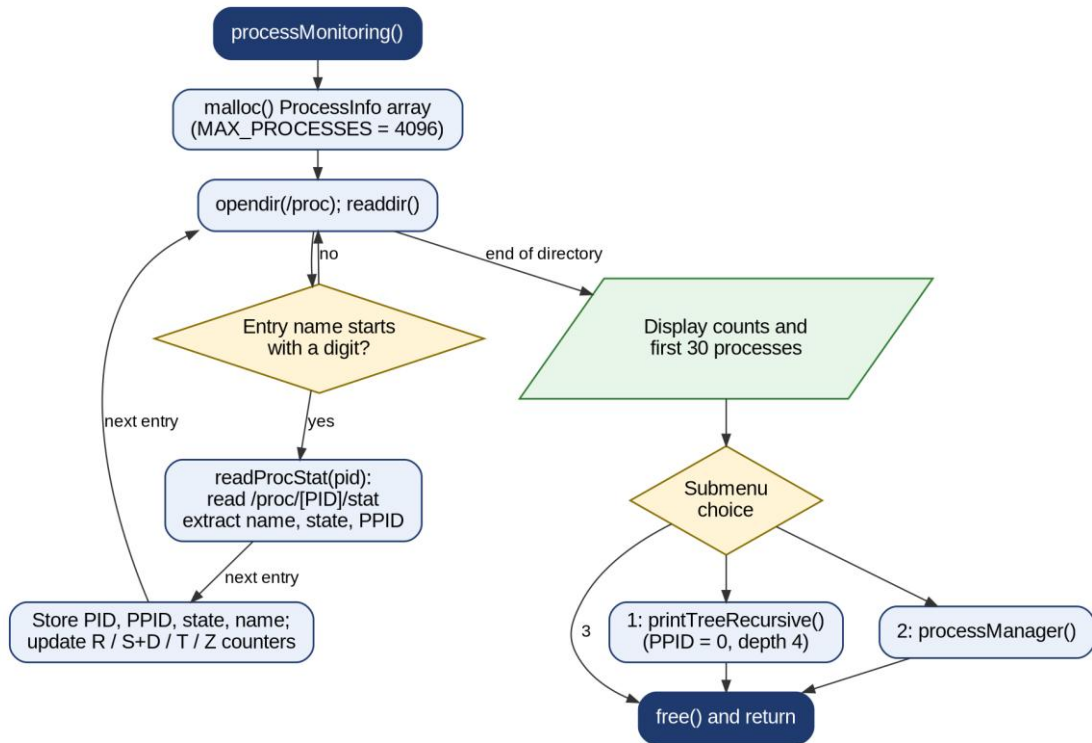


Figure 6.4: Process monitoring flow

Figure 6.5 shows a process hierarchy built from the process list captured in Figure 7.6. It is an illustration of how the PID and PPID values of that list form a tree, with PID 1 at the root; the monitor program itself appears as PID 525, started from the bash shell (PID 335).

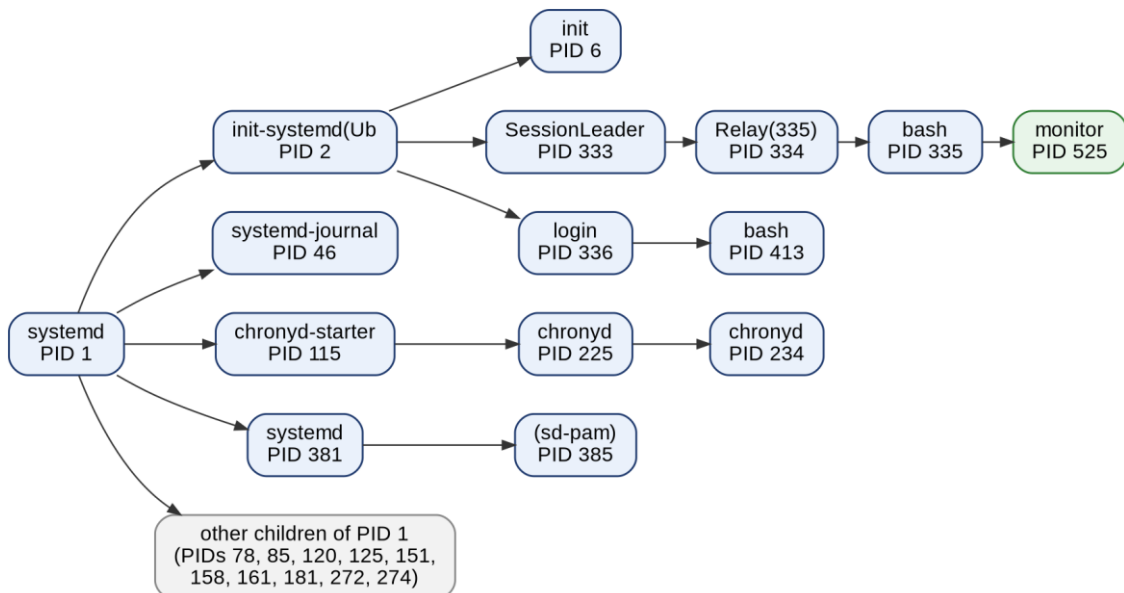


Figure 6.5: Process hierarchy diagram based on the process list in Figure 7.6

6.1.6 System Uptime

The function `systemUptime()` reads `/proc/uptime`, which contains two numbers: the time in seconds since the system booted, and the cumulative time in seconds that all processors have spent idle. The uptime in seconds is converted as follows:

```
days    = t / 86400
hours    = (t % 86400) / 3600
minutes  = (t % 3600) / 60
seconds  = t % 60
```

The screen shows the formatted uptime in the form days, hours, minutes and seconds, the total uptime in hours and in seconds, and the cumulative idle time in hours. The menu entry for this module refers to cumulative CPU time; the value displayed is the cumulative idle time given by `/proc/uptime`. On a system with several logical processors, this idle value can be larger than the uptime itself because it is summed over all processors.

6.1.7 System Health Summary

The function `systemHealthSummary()` calls `getCPUUsage()`, `getMemoryUsage()`, `getDiskUsage()` (root file system) and `getProcessCounts()`. It displays three progress gauges for CPU load, RAM and the root disk, and the process counts (total and running). The composite status is determined by the highest of the three percentages:

```
worst = max(cpu, memory, disk)
composite status = badge(worst)    /* NORMAL < 70, WARNING >= 70, CRITICAL >= 85 */
```

Process counts are displayed for information but do not influence the composite status. If one of the readings cannot be obtained, its gauge is not shown.

6.1.8 Network Monitoring

The helper `readNetDev()` opens `/proc/net/dev`. Every line that contains a colon belongs to a network interface: the text before the colon is the interface name, and the numbers after the colon are the receive and transmit counters. The function reads nine numbers; the first is the number of bytes received, the next seven are other receive counters that are skipped, and the ninth is the number of bytes transmitted. Up to 16 interfaces are stored in an array of `NetInterface` structures (name, rxBytes, txBytes).

The function `networkMonitoring()` displays a table with the received and transmitted amounts, converted by `formatBytes()` into B, KB, MB, GB or TB using a factor of 1024. The loopback interface `lo` is drawn in grey. These counters are cumulative values since the interface was created, not speeds.

If the user selects option 0, a live bandwidth monitor starts. The program stores a first reading, waits up to one second for a key press, reads the counters again, clears the screen and prints for each interface:

```
Download speed (KB/s) = (RX bytes now - RX bytes before) / 1024
Upload speed   (KB/s) = (TX bytes now - TX bytes before) / 1024
```

The previous reading is then replaced by the current one. The interval is taken as approximately one second; it is not measured with a clock, so the displayed speeds are approximate. The monitor stops with q, ESC or End.

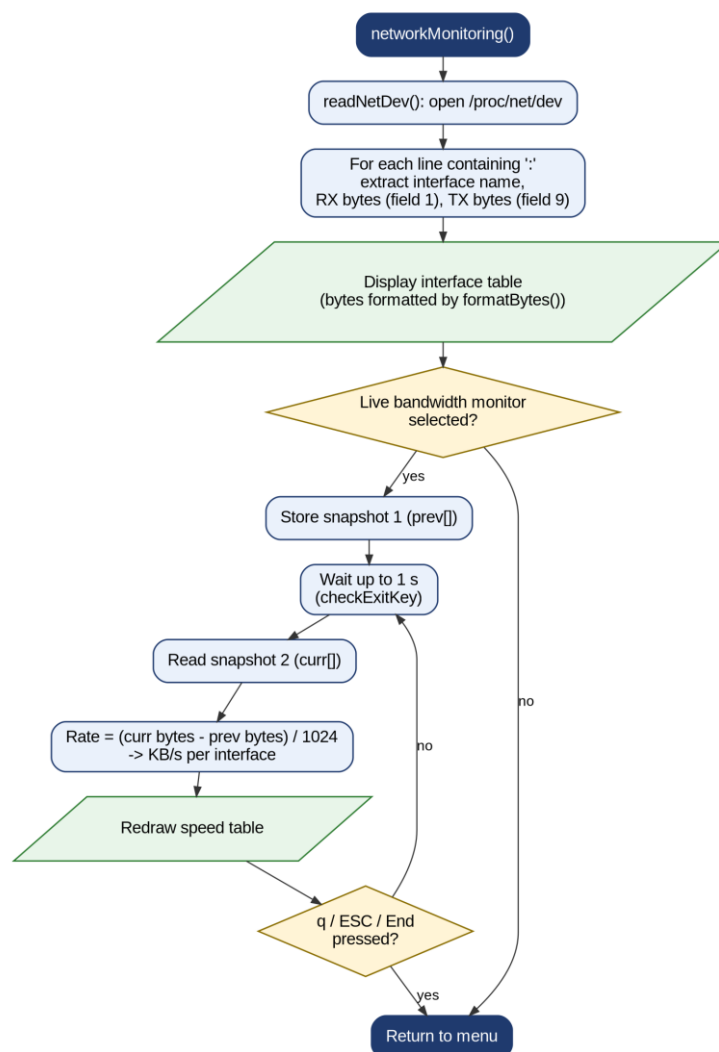


Figure 6.6: Network monitoring flow

6.1.9 Top Resource-Consuming Processes

The function `topResourceProcesses()` finds the processes that use the most CPU and memory. It allocates an array of `ResourceProcess` structures and an array for the first CPU-time readings, both for up to 4096 processes. The steps are:

1. For each numeric directory in /proc, read utime + stime (in clock ticks) from /proc/[PID]/stat and store it with the PID and name.
2. Read the total CPU time of the system with getTotalCPUTime() (the sum of the eight counters of /proc/stat), wait 300 ms, and read it again.
3. For each stored process read utime + stime again. The CPU usage of the process is its increase in CPU time divided by the increase in total CPU time, multiplied by 100.
4. For each process read /proc/[PID]/statm. The second field is the resident set size in pages; it is multiplied by the page size (sysconf(_SC_PAGESIZE)) and divided by 1024 to obtain kilobytes, and by 1024 again to obtain megabytes.
5. Sort the array with qsort() in descending order of CPU usage using the comparison function cmpCPU(), and print the first ten entries with a small bar.
6. Sort the same array with qsort() in descending order of resident memory using cmpMem(), and print the first ten entries in megabytes.

Since the denominator is the total CPU time of all logical processors, the CPU percentage of a process is its share of the whole processor capacity during the 300 ms interval. A process that has ended during the interval, or whose file cannot be read, is shown with 0 %. Memory is reported as resident set size (RSS), that is, the physical memory currently held by the process.

6.1.10 Process Search

The function processSearch() asks for a process name or PID. The input is read with fgets() and trailing line-end characters are removed; an empty input returns immediately. The function then scans /proc in the same way as the process monitor and reads PID, PPID, state and name of each process. A process matches if its name contains the entered text, ignoring case (strcasestr()), or if its PID is exactly equal to the entered text. Matching processes are printed in a table of PID, PPID, STATE and PROCESS NAME, followed by the number of matches. If there is no match, a message is shown. If exactly one process is found, the user is asked whether to manage it, and choosing 1 opens the process manager.

6.1.11 Resource Alerts

The function resourceAlerts() obtains CPU usage, RAM usage and root-disk usage with getCPUUsage(), getMemoryUsage() and getDiskUsage(). For each of them it prints the value with a status badge and, depending on the value, an alert or notice message (Table 6.2).

Table 6.2: Alert thresholds used by resourceAlerts()

Resource	Condition	Message
CPU	Above 85 %	ALERT: Critical CPU load!

Resource	Condition	Message
CPU	Above 70 % up to 85 %	NOTICE: Elevated CPU load.
RAM	Above 85 %	ALERT: Memory pressure critical!
RAM	Above 70 % up to 85 %	NOTICE: High memory pressure.
Disk (/)	Above 85 %	ALERT: Low disk space!
Disk (/)	Above 70 % up to 85 %	NOTICE: Root disk filling up.
All	70 % or below	No message; badge NORMAL

If none of the three resources raises an alert or notice, the message “All system metrics are operating within safe parameters” is displayed. The threshold values 70 and 85 are fixed in the source code. The messages use a strict “greater than” comparison, whereas the badge uses “greater than or equal”, so a reading of exactly 70.0 % or 85.0 % shows the higher badge without the corresponding message. Network usage is not part of the alert module.

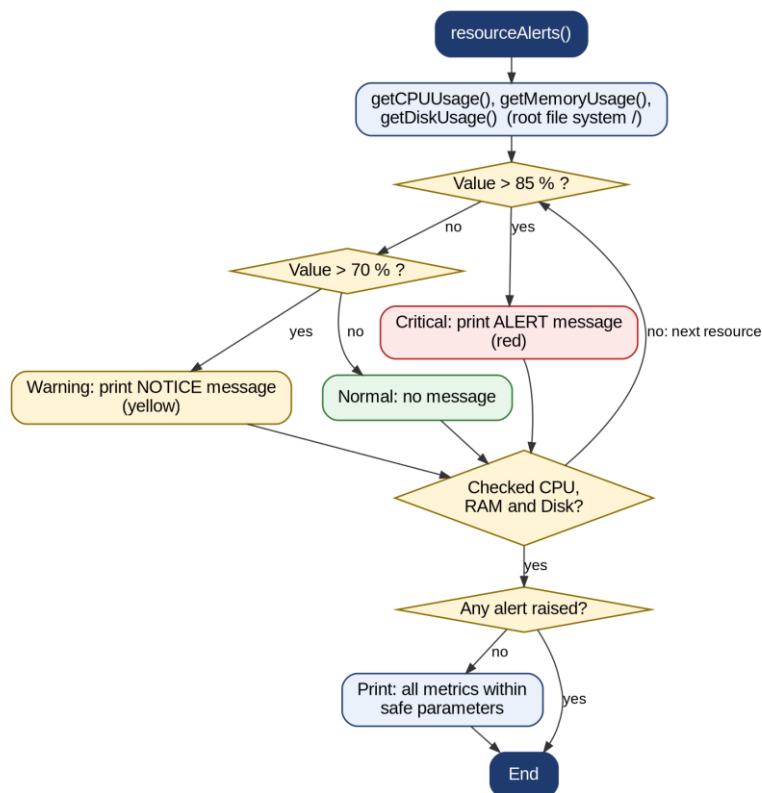


Figure 6.7: Resource alert flow

6.1.12 Resource Logging

The function resourceLogging() obtains the CPU, RAM and root-disk usage, creates a timestamp with time(), localtime() and strftime() in the format YYYY-MM-DD HH:MM:SS, and opens the file monitor_log.txt (defined by the macro LOG_FILE) in append mode with fopen(). One line is written with fprintf():

If a reading could not be obtained, the value -1.0 is written for it. The file is created in the current working directory, and new entries are added at the end so earlier entries are kept. One entry is written each time the option is selected; the program does not log automatically at intervals. After writing, the screen shows the file name, the timestamp and the recorded values, and offers to display the last ten entries. To show them, the file is read line by line and the most recent ten lines are kept in a circular buffer indexed by the line count modulo 10.

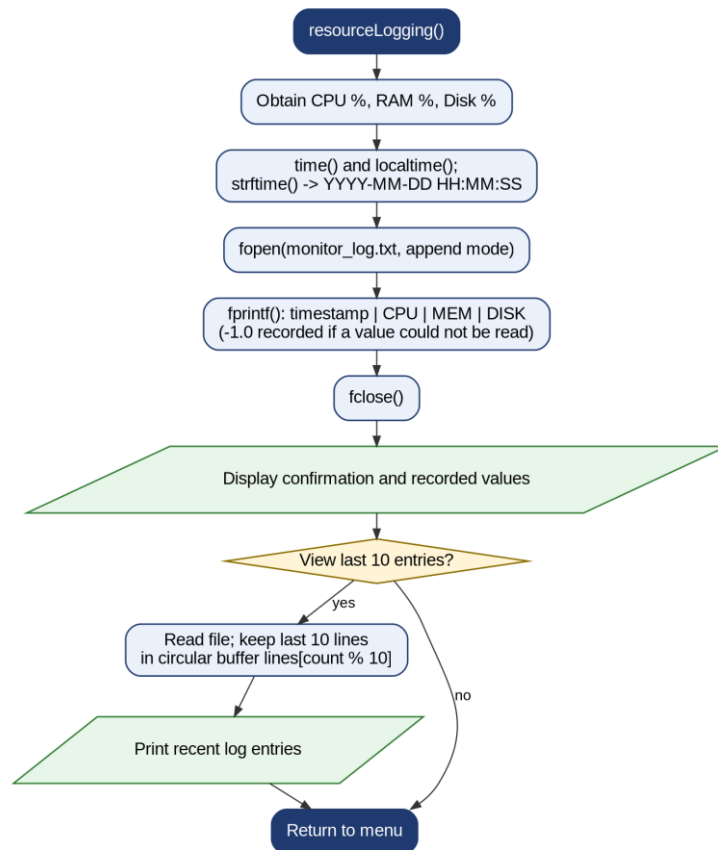


Figure 6.8: Resource logging flow

6.1.13 Interactive Process Manager

The process manager is implemented in `processManager()`. It can be opened from menu option 11, from the submenu of the process monitor, or from the process search when a single match is found. The function reads a PID with `scanf()`, rejects values that are not positive, and checks with `access()` that the directory `/proc/[PID]` exists. It then offers six actions:

Table 6.3: Actions of the process manager

No.	Action	System call
1	Graceful terminate (SIGTERM, 15)	kill(pid, SIGTERM)
2	Force kill (SIGKILL, 9)	kill(pid, SIGKILL)

No.	Action	System call
3	Pause process (SIGSTOP, 19)	kill(pid, SIGSTOP)
4	Resume process (SIGCONT, 18)	kill(pid, SIGCONT)
5	Change priority (renice)	setpriority(PRIO_PROCESS, pid, value), value from -20 to 19
6	Cancel	No action

The result of each call is reported: success is confirmed, and on failure the message from `strerror(errno)` is printed. Whether an operation is allowed depends on the Linux permission rules; for example, a normal user cannot signal processes of other users or lower the nice value below 0, and the program mentions that `sudo` is needed for negative nice values. The function does not ask for a confirmation before sending a signal.

6.1.14 System Load Averages and Context Switches

The function `systemLoadAndScheduling()` reads `/proc/loadavg`, which contains the load averages of the last 1, 5 and 15 minutes, the number of currently runnable and total kernel scheduling entities (in the form `r/t`), and the last process ID that was assigned. Each load average is divided by the number of online cores (`sysconf(_SC_NPROCESSORS_ONLN)`) and shown as a percentage per core with a gauge limited to 100 %. The function then reads `/proc/stat` again to obtain the line values `ctxt` (total context switches since boot), `processes` (total forks since boot), `procs_running` (processes in the run queue) and `procs_blocked` (processes blocked on I/O), and prints them.

6.1.15 Kernel Modules Explorer

The function `kernelModulesExplorer()` reads `/proc/modules`. Each line gives the module name, its size in bytes, the number of instances using it and its state. The function counts all modules, adds up their sizes, and stores the first 20 in a local array of a small structure (`ModuleEntry`). It prints the total number of modules and their total memory, followed by a table of the first 20 modules. The user can then enter a substring, and the file is read again to print all modules whose name contains it (`strcasestr()`, case-insensitive). This module only reads information; it does not load or unload kernel modules. If `/proc/modules` cannot be read, an error message is printed.

6.2 Algorithm / Pseudocode

The pseudocode below follows the corresponding functions of `main.c`.

Algorithm 1: CPU usage calculation (getCPUUsage)

```

READ first line of /proc/stat -> u1, n1, s1, i1, io1, ir1, sir1, st1
IF reading or parsing fails THEN RETURN failure
WAIT 300 ms
READ first line of /proc/stat -> u2, n2, s2, i2, io2, ir2, sir2, st2
total1 <- u1 + n1 + s1 + i1 + io1 + ir1 + sir1 + st1
total2 <- u2 + n2 + s2 + i2 + io2 + ir2 + sir2 + st2

```

```

idle1 <- i1 + io1 ; idle2 <- i2 + io2
dTotal <- total2 - total1 ; dIdle <- idle2 - idle1
IF dTotal > 0 THEN usage <- 100 * (dTotal - dIdle) / dTotal
ELSE usage <- 0
RETURN usage

```

Algorithm 2: Memory usage calculation (getMemoryUsage / memoryUsage)

```

OPEN /proc/meminfo
FOR each line: IF line starts with "MemTotal:" THEN total <- value
                IF line starts with "MemAvailable:" THEN avail <- value
                (memoryUsage() also reads Cached, Buffers, Shmem, Active,
                 Inactive, SwapTotal, SwapFree)
IF total <= 0 THEN RETURN failure
used <- total - avail
ramPct <- used / total * 100
swapUsed <- swapTotal - swapFree
IF swapTotal > 0 THEN swapPct <- swapUsed / swapTotal * 100

```

Algorithm 3: Disk usage calculation (diskUsage / setDiskMount)

```

OPEN /proc/mounts ; count <- 0
FOR each line while count < 32:
    READ device, mount, fstype
    IF device starts with "/dev/" OR mount = "/" THEN
        IF statvfs(mount) succeeds AND f_blocks > 0 AND mount not already stored
        THEN
            total <- f_blocks * f_frsize ; free <- f_bavail * f_frsize
            used <- total - free ; usage <- used / total * 100
            STORE (device, mount, fstype, total, used, free, usage)
IF count = 0 THEN use statvfs("/") as the only entry
PRINT table and a progress bar for each stored mount

```

Algorithm 4: Process scanning (processMonitoring)

```

ALLOCATE array procs[4096]
OPEN directory /proc
FOR each entry while count < 4096:
    IF first character of the name is a digit THEN
        pid <- number(name)
        IF readProcStat(pid) succeeds THEN
            STORE pid, ppid, state, name
            IF state = R THEN running++
            ELSE IF state = S or D THEN sleeping++
            ELSE IF state = T THEN stopped++
            ELSE IF state = Z THEN zombie++
        count++
PRINT counts and first 30 entries
FREE procs

```

Algorithm 5: Process tree construction (printTreeRecursive)

```

PROCEDURE PrintTree(procs, count, parentPid, level, maxDepth)
    IF level > maxDepth THEN RETURN
    FOR i <- 0 TO count - 1:
        IF procs[i].ppid = parentPid THEN
            PRINT indentation(level), procs[i].name, procs[i].pid
            PrintTree(procs, count, procs[i].pid, level + 1, maxDepth)
CALL PrintTree(procs, count, 0, 0, 4)

```

Algorithm 6: Top CPU process calculation (topResourceProcesses)

```

FOR each numeric directory in /proc:
    IF utime + stime can be read THEN STORE pid, name, cpuTime1
total1 <- total CPU time ; WAIT 300 ms ; total2 <- total CPU time
dTotal <- total2 - total1 (use 1 if not positive)
FOR each stored process:
    read cpuTime2
    IF read succeeds AND cpuTime2 >= cpuTime1 THEN

```

```

    cpu% <- 100 * (cpuTime2 - cpuTime1) / dTotal
  ELSE cpu% <- 0
  SORT processes by cpu% in descending order  (qsort, cmpCPU)
  PRINT first 10

```

Algorithm 7: Top memory process calculation

```

FOR each stored process:
  READ residentPages from /proc/[PID]/statm (second field)
  memoryKB <- residentPages * pageSize / 1024
  memoryMB <- memoryKB / 1024
  SORT processes by memoryKB in descending order  (qsort, cmpMem)
  PRINT first 10

```

Algorithm 8: Process search (processSearch)

```

READ term ; remove line end ; IF term is empty THEN RETURN
found <- 0
FOR each numeric directory in /proc:
  IF readProcStat(pid) succeeds THEN
    IF name contains term (ignoring case) OR text(pid) = term THEN
      PRINT pid, ppid, state, name ; found++
IF found = 0 THEN PRINT "No matching processes"
ELSE PRINT found ; IF found = 1 THEN offer the process manager

```

Algorithm 9: Resource alert detection (resourceAlerts)

```

cpu <- getCPUUsage() ; mem <- getMemoryUsage() ; disk <- getDiskUsage("/")
alert <- false
FOR each resource value v that was read successfully:
  PRINT v and badge(v)
  IF v > 85 THEN PRINT ALERT message ; alert <- true
  ELSE IF v > 70 THEN PRINT NOTICE message ; alert <- true
IF alert = false THEN PRINT "All system metrics are operating within safe
parameters"

```

Algorithm 10: Resource logging (resourceLogging)

```

cpu, mem, disk <- current readings (-1.0 if a reading fails)
timestamp <- local time formatted as YYYY-MM-DD HH:MM:SS
OPEN monitor_log.txt in append mode ; IF it cannot be opened THEN RETURN
WRITE "[timestamp] CPU: cpu% | MEM: mem% | DISK: disk%" ; CLOSE file
PRINT confirmation
IF user chooses to view history THEN
  READ file line by line into circular buffer of 10 lines
  PRINT the last 10 lines

```

6.3 Core C Implementation

6.3.1 Data Structures

Four structures are defined at the beginning of main.c (Table 6.4).

Table 6.4: Data structures of main.c

Structure	Members	Used by
ProcessInfo	int pid; int ppid; char state; char name[256]	processMonitoring(), printTreeRecursive()
ResourceProcess	int pid; char name[256]; double cpuUsage; double memoryUsage; unsigned long long memoryKB	topResourceProcesses(), cmpCPU(), cmpMem()
NetInterface	char name[64]; unsigned long long rxBytes; unsigned long long txBytes	readNetDev(), networkMonitoring()

Structure	Members	Used by
DiskMount	char mountPoint[256]; char device[128]; char fsType[32]; double totalGB, usedGB, freeGB, usagePercent	setDiskMount(), diskUsage()

The function `kernelModulesExplorer()` also declares a small local structure (`ModuleEntry`) with the module name, size, instance count and state.

6.3.2 Functions

Table 6.5: Main functions of main.c

Function	Role
<code>main()</code>	Displays the menu, reads the choice and calls the module through a switch statement
<code>systemInformation()</code>	Module 1: system and processor information
<code>getCPUUsage()</code> , <code>getTotalCPUTime()</code> , <code>cpuUsage()</code>	CPU percentage, total CPU time, CPU module with live mode
<code>getMemoryUsage()</code> , <code>memoryUsage()</code>	RAM percentage; memory module
<code>setDiskMount()</code> , <code>getDiskUsage()</code> , <code>diskUsage()</code>	Disk calculation for a mount; root-disk percentage; disk module
<code>readProcStat()</code>	Reads and parses <code>/proc/[PID]/stat</code> (name, state, PPID, CPU time)
<code>getProcessCounts()</code> , <code>processMonitoring()</code> , <code>printTreeRecursive()</code>	Process counts; process module; process tree
<code>systemUptime()</code> , <code>systemHealthSummary()</code>	Uptime; health summary
<code>readNetDev()</code> , <code>networkMonitoring()</code>	Network counters; network module with live speed
<code>readProcessMemory()</code> , <code>cmpCPU()</code> , <code>cmpMem()</code> , <code>topResourceProcesses()</code>	Resident memory of a process; sort comparators; top-process module
<code>processSearch()</code> , <code>processManager()</code>	Process search; signals and priority
<code>systemLoadAndScheduling()</code> , <code>kernelModulesExplorer()</code>	Load averages and scheduler counters; kernel modules
<code>resourceAlerts()</code> , <code>resourceLogging()</code> , <code>afterFeature()</code>	Alerts; logging; return-to-menu prompt
<code>enableRawMode()</code> , <code>disableRawMode()</code> , <code>checkExitKey()</code>	Terminal raw mode and non-blocking key check for live monitors
<code>handle_sigint()</code> , <code>setupSignalHandler()</code>	Restore the terminal on SIGINT/SIGTERM and at exit
<code>printHeader()</code> , <code>printSection()</code> , <code>printDivider()</code> , <code>printProgressBar()</code> , <code>printStatusBadge()</code> , <code>formatBytes()</code> , <code>clearScreen()</code>	Output formatting helpers

6.3.3 Selected Code Snippets

The following extracts are taken from main.c. Long parts are shortened with comments for readability.

System information (systemInformation)

```
struct utsname info;
long cores = sysconf(_SC_NPROCESSORS_ONLN);
if (uname(&info) != 0) return;
...
FILE *cpuf = fopen("/proc/cpuinfo", "r");
while (fgets(line, sizeof(line), cpuf)) {
    if (strncmp(line, "model name", 10) == 0 || strncmp(line, "Processor", 9) == 0)
    {
        char *c = strchr(line, ':');
        if (c) { c += 2; c[strcspn(c, "\r\n")] = '\0';
            snprintf(cpuModel, sizeof(cpuModel), "%s", c); }
        } else if (strncmp(line, "cpu MHz", 7) == 0) { /* parse frequency */ }
    }
    ...
    struct passwd *pw = getpwuid(getuid());
```

uname() fills the structure with kernel information, and the loop searches /proc/cpuinfo for the lines that start with the required keywords.

CPU calculation (getCPUUsage)

```
usleep(300000); /* 300ms responsive sampling */
...
unsigned long long t1 = u1 + n1 + s1 + i1 + io1 + ir1 + sir1 + st1;
unsigned long long t2 = u2 + n2 + s2 + i2 + io2 + ir2 + sir2 + st2;
unsigned long long idle1 = i1 + io1, idle2 = i2 + io2;
unsigned long long dt = t2 - t1, didle = idle2 - idle1;

*usage = (dt > 0) ? (100.0 * (double)(dt - didle) / (double)dt) : 0.0;
```

The two samples of /proc/stat are reduced to total and idle times, and the usage is the non-idle part of the difference.

Memory calculation (getMemoryUsage)

```
while (fgets(line, sizeof(line), f)) {
    if (strncmp(line, "MemTotal:", 9) == 0) sscanf(line, "MemTotal: %ld kB",
    &totalKB);
    else if (strncmp(line, "MemAvailable:", 13) == 0) sscanf(line, "MemAvailable:
    %ld kB", &availKB);
}
if (totalKB <= 0) return 0;
*usage = ((double)(totalKB - availKB) / (double)totalKB) * 100.0;
```

Disk calculation (setDiskMount)

```
unsigned long long total = (unsigned long long)st->f_blocks * st->f_frsize;
unsigned long long freeB = (unsigned long long)st->f_bavail * st->f_frsize;
unsigned long long usedB = total > freeB ? total - freeB : 0;
m->totalGB = (double)total / (1024.0 * 1024.0 * 1024.0);
m->usagePercent = total > 0 ? (100.0 * (double)usedB / (double)total) : 0.0;
```

Process scanning (readProcStat and processMonitoring)

```
char *lp = strchr(line, '(');
char *rp = strrchr(line, ')');
if (!lp || !rp || rp <= lp) return 0;
/* copy the text between the parentheses into name */
```

```

char *after = rp + 2;
sscanf(after, "%c %d", &st, &parent); /* state and PPID */
...
while ((e = readdir(dir)) != NULL && count < MAX_PROCESSES) {
    if (!isdigit((unsigned char)e->d_name[0])) continue;
    int pid = atoi(e->d_name);
    if (readProcStat(pid, procs[count].name, MAX_NAME, &st, &ppid, NULL)) {
        procs[count].pid = pid; procs[count].ppid = ppid; procs[count].state = st;
        if (st == 'R') rCount++;
        else if (st == 'S' || st == 'D') sCount++;
        else if (st == 'T') tCount++;
        else if (st == 'Z') zCount++;
        count++;
    }
}

```

Process tree (printTreeRecursive)

```

static void printTreeRecursive(ProcessInfo *processes, int count,
                               int parentPid, int level, int maxDepth) {
    if (level > maxDepth) return;
    for (int i = 0; i < count; i++) {
        if (processes[i].ppid == parentPid) {
            for (int j = 0; j < level; j++) printf(" ");
            ...
            printTreeRecursive(processes, count, processes[i].pid, level + 1,
maxDepth);
        }
    }
}

```

Network monitoring (readNetDev)

```

char *col = strchr(line, ':');
if (!col) continue;
*col = '\0';
sscanf(line, "%63s", ifname);
unsigned long long rx = 0, tx = 0, dummy[7];
if (sscanf(col + 1, "%llu %llu %llu %llu %llu %llu %llu %llu",
            &rx, &dummy[0], &dummy[1], &dummy[2], &dummy[3],
            &dummy[4], &dummy[5], &dummy[6], &tx) == 9) { /* store name, rx, tx */ }

```

Logging (resourceLogging)

```

FILE *f = fopen(LOG_FILE, "a");
if (!f) return;
fprintf(f, "[%s] CPU: %5.1f%% | MEM: %5.1f%% | DISK: %5.1f%%\n",
        timeStr, cOK ? cpu : -1.0, mOK ? mem : -1.0, dOK ? disk : -1.0);
fclose(f);

```

6.4 Linux System Calls / APIs Used

Table 6.6: Linux system calls and APIs used in main.c

Function / API	Purpose	Module
uname()	Kernel name, release, architecture, host name	System Information
sysconf(_SC_NPROCESSORS_ONLN)	Number of online logical processors	System Information, Load
sysconf(_SC_PAGESIZE)	Page size for converting RSS pages to KB	Top Processes
getuid(), getpwuid(), getenv()	Current user name and home directory	System Information
statvfs()	File-system block statistics	Disk, Health, Alerts, Logging
opendir(), readdir(), closedir()	Scan /proc for process directories	Processes, Top Processes, Search

Function / API	Purpose	Module
fopen(), fgets(), fscanf(), fclose()	Read /proc files and /etc/os-release	All monitoring modules
sscanf(), snprintf()	Parse text lines; build strings	All monitoring modules
strncmp(), strchr(), strrchr(), strcspn(), memcpy()	String comparison and extraction	System Information, Processes
strcasestr()	Case-insensitive substring search	Process Search, Kernel Modules
qsort()	Sort processes by CPU and by memory	Top Processes
malloc(), free()	Dynamic process arrays	Processes, Top Processes
usleep(), sleep()	Sampling delay; pauses between screens	CPU, Top Processes, Network, main
select(), read()	Wait for a key press with a time limit	Live CPU and network monitors
tcgetattr(), tcsetattr()	Switch terminal between normal and raw mode	Live CPU and network monitors
signal(), atexit()	Install SIGINT/SIGTERM handler; restore terminal at exit	Program start-up
kill()	Send SIGTERM, SIGKILL, SIGSTOP, SIGCONT	Process Manager
setpriority()	Change the nice value of a process	Process Manager
access()	Check that /proc/[PID] exists	Process Manager
time(), localtime(), strftime()	Create the log timestamp	Resource Logging
fprintf()	Write the log line	Resource Logging
isdigit(), atoi()	Recognise and convert PID directory names	Processes, Search, Top Processes
strerror(), errno	Report why a call failed	Process Manager

6.5 Process / Thread Implementation

The application is a single-threaded, sequential program. It does not call `fork()`, `exec()` or any `pthread` function, so it creates neither child processes nor worker threads. It does not need to: its task is to analyse processes that already exist. Process monitoring is therefore implemented by reading the per-process directories of `/proc`, as described in Sections 6.1.5, 6.1.9 and 6.1.10, and by controlling other processes through `kill()` and `setpriority()` in the process manager.

The monitor program itself is also a process and appears in its own process list. In the process list captured in Figure 7.6 it is listed with the command name `monitor` and the state `R` (running), because it is the process that is executing while it reads `/proc`. The fields shown for threads in some outputs, such as “Runnable Threads” in the `load` module, are values reported by the kernel in `/proc/loadavg` and do not mean that the program creates threads.

6.6 IPC / Synchronization Implementation

The current version of the project does not implement explicit IPC mechanisms such as pipes, shared memory, message queues, or semaphores. The project focuses primarily on system monitoring and resource management.

Because the program has a single flow of control, it also needs no synchronisation primitives such as mutexes or semaphores. The program does use the Linux signal mechanism in two limited ways: it installs a handler for SIGINT and SIGTERM, which restores the terminal settings before the program exits, and the process manager sends signals to other processes with kill(). These uses are for terminal clean-up and process control. They are not a communication channel designed between cooperating processes of this project, and therefore CO3 is not claimed as implemented.

6.7 Memory Management Implementation

Memory management in the project has two aspects: obtaining memory information from the operating system, and managing the program's own memory.

Memory information. RAM, available memory, cached memory, buffers, shared memory and swap are read from /proc/meminfo (Section 6.1.3), and the resident memory of each process is read from /proc/[PID]/statm and converted using the system page size (Section 6.1.9). The used memory is calculated from MemTotal and MemAvailable. MemAvailable is the kernel's estimate of the memory that can be given to new applications without swapping, so it is more meaningful than the simple free memory value because it includes reclaimable cache.

Program memory. The project does not implement a custom memory manager. Memory is used in two ways. Most data is held in fixed-size local arrays and buffers on the stack, for example the 32 DiskMount entries, 16 NetInterface entries and 20 kernel-module entries. The larger process arrays are allocated dynamically with malloc() and released with free() before the function returns (Table 6.7). The fixed sizes mean that the program ignores any entries beyond these limits.

Table 6.7: Dynamic memory allocation in main.c

Function	Allocation	Approximate size
processMonitoring()	Array of 4096 ProcessInfo	1.0 MB
topResourceProcesses()	Array of 4096 ResourceProcess	1.1 MB
topResourceProcesses()	Array of 4096 unsigned long long (first CPU readings)	32 KB

6.8 User Interface Implementation

Terminal interface and menu. The user interface is text-based. The function `main()` runs an infinite loop that clears the screen with ANSI escape sequences, prints the title and the 16 numbered options in three groups, and reads the choice with `scanf()`. A switch statement calls the function of the selected module (Figure 7.1).

Validation. If `scanf()` does not read an integer, the message “Invalid input. Please enter a valid number.” is printed, the rest of the line is discarded, and the menu is shown again after a one-second pause. If the number is outside 1–16, the message “Invalid choice. Please select between 1 and 16.” is shown. Submenus and process-manager inputs are checked in a similar way; a PID must be a positive number and must exist in `/proc`, and a nice value must be between -20 and 19.

Output formatting. The helper functions `printHeader()`, `printSection()` and `printDivider()` draw headers and separators; `printProgressBar()` draws a bar with the characters `█` and `░` whose colour depends on the value (green, yellow, red); `printStatusBadge()` prints NORMAL, WARNING or CRITICAL; and `formatBytes()` converts byte counts to readable units. Colours are produced with ANSI escape sequences, so the terminal must support them. The progress bars use Unicode characters, so a UTF-8 capable terminal is also needed.

Live monitors. The live CPU and network monitors switch the terminal to raw mode with `tcsetattr()` by disabling canonical mode and echo, so that single key presses can be read without waiting for the Enter key. `checkExitKey()` uses `select()` with a timeout to wait for a key and returns true for q, Q, x, X, ESC (including the End key sequence). When the monitor ends, the original terminal settings are restored.

Return to menu and exit. After a module finishes, `afterFeature()` asks the user to enter 1 to return to the main menu or 2 to exit. Option 16 of the main menu also exits. If the user presses Ctrl+C or the process receives SIGTERM, the handler restores the terminal settings, prints “Process interrupted.” and ends the program.

CHAPTER 7

EXECUTION AND DEMONSTRATION

This chapter explains how the program is compiled and run, and demonstrates each module. Screenshots that were captured from the running application are included without modification. Where a screenshot has not yet been inserted, a placeholder is shown; the explanation next to it is based on the behaviour defined in the source code.

7.1 Compilation in Ubuntu

Open the Ubuntu terminal, change to the directory that contains main.c and compile the program with GCC:

```
cd ~/linux-system-monitor  
gcc -Wall -Wextra main.c -o monitor
```

If there are no errors, the executable file monitor is created in the same directory.

7.2 Execution Commands

```
./monitor
```

The program should be run in a terminal that supports ANSI colours and UTF-8 characters, because the interface uses colour codes and block characters for the progress bars. Most features can be used as a normal user. Sending signals to processes of other users and setting a negative nice value require administrator privileges (sudo). The log file monitor_log.txt is created in the directory from which the program is started.

7.3 Main Menu

```
=====
                        LINUX SYSTEM MONITOR
=====

[ SYSTEM & HARDWARE METRICS ]
 1. System Information & OS Details
 2. CPU Usage & Live Frequency Monitor
 3. Memory Utilization (RAM & Swap)
 4. Disk & Multi-Mount Storage
 5. Running Processes & Process Tree
 6. System Uptime & Cumulative CPU Time
 7. System Health Summary Dashboard

[ PROCESS & NETWORK MANAGEMENT ]
 8. Network Monitoring & Live Bandwidth
 9. Top Resource-Consuming Processes
10. Process Search & Filter
11. Interactive Process Manager (Kill/Pause/Nice)

[ KERNEL DIAGNOSTICS & TELEMETRY ]
12. System Load Averages & Context Switches
13. Kernel Modules Explorer (Loaded LKMs)
14. Resource Threshold Alerts
15. Resource Usage Logging & History
16. Exit
```

Figure 7.1: Main menu of the Linux System Information and Resource Monitoring Tool

Figure 7.1 shows the main menu. The title “LINUX SYSTEM MONITOR” is followed by three groups of options: System and Hardware Metrics (1–7: system information, CPU usage, memory utilisation, disk and multi-mount storage, running processes and process tree, system uptime, system health summary), Process and Network Management (8–11: network monitoring and live bandwidth, top resource-consuming processes, process search and filter, interactive process manager) and Kernel Diagnostics and Telemetry (12–15 and Exit: load averages and context switches, kernel modules explorer, resource threshold alerts, resource usage logging and history, and exit). The user enters the number of the required feature.

7.4 System Information

```
=====
  SYSTEM INFORMATION
=====

--- OS & KERNEL DETAILS ---
Distribution      : Ubuntu 26.04 LTS
Kernel           : Linux (6.18.33.2-microsoft-standard-WSL2)
Architecture     : x86_64
Hostname         : VSK-HP

--- PROCESSOR SPECIFICATIONS ---
CPU Model        : 13th Gen Intel(R) Core(TM) i5-1335U
Logical Cores    : 12 Cores (Online)
Frequency        : 2496.01 MHz

--- CURRENT USER SESSION ---
Username         : venkatsaikrishna5 (UID: 1000)
Home Directory   : /home/venkatsaikrishna5
=====
```

Figure 7.2: System information display

Figure 7.2 shows the System Information screen. It displays the OS and kernel details (distribution Ubuntu 26.04 LTS, kernel Linux 6.18.33.2-microsoft-standard-WSL2, architecture x86_64, host name VSK-HP), the processor specification (13th Gen Intel Core i5-1335U, 12 logical cores online, frequency 2496.01 MHz) and the current user session (user name, UID 1000 and home directory). The kernel string shows that the screenshot was taken on Ubuntu running under WSL2. The information comes from `uname()`, `/etc/os-release`, `/proc/cpuinfo`, `sysconf()` and `getpwuid()`.

7.5 CPU Monitoring

```
=====
  CPU USAGE & REAL-TIME MONITOR
=====

Overall CPU      [ ] 0.0%
[NORMAL]

Options:
  0. Start Real-Time Live CPU Monitor
  1. Return to Main Menu
Enter choice: 0

Live monitoring active. Press 'q', ESC, or End to stop.
[LIVE] CPU Usage [ ] 0.0%
[LIVE] CPU Usage [ ] 0.0%
[LIVE] CPU Usage [ ] 0.0%
[LIVE] CPU Usage [ ] 0.0%
[LIVE] CPU Usage [ ] 0.3%
[LIVE] CPU Usage [ ] 0.0%
[LIVE] CPU Usage [ ] 0.0%

Real-time monitoring stopped.
```

Figure 7.3: CPU usage monitoring

Option 2 measures CPU utilisation by sampling `/proc/stat` twice, 300 ms apart, and shows the result as a progress bar with a status badge (NORMAL, WARNING or CRITICAL). The submenu then offers option 0, the live monitor. In the live monitor the CPU line is rewritten about every 0.7 seconds, so the value changes as the load on the system changes, until the user presses q, ESC or End. This demonstrates sampled monitoring of the CPU time counters; the values depend on what the system is doing at that moment.

7.6 Memory Monitoring

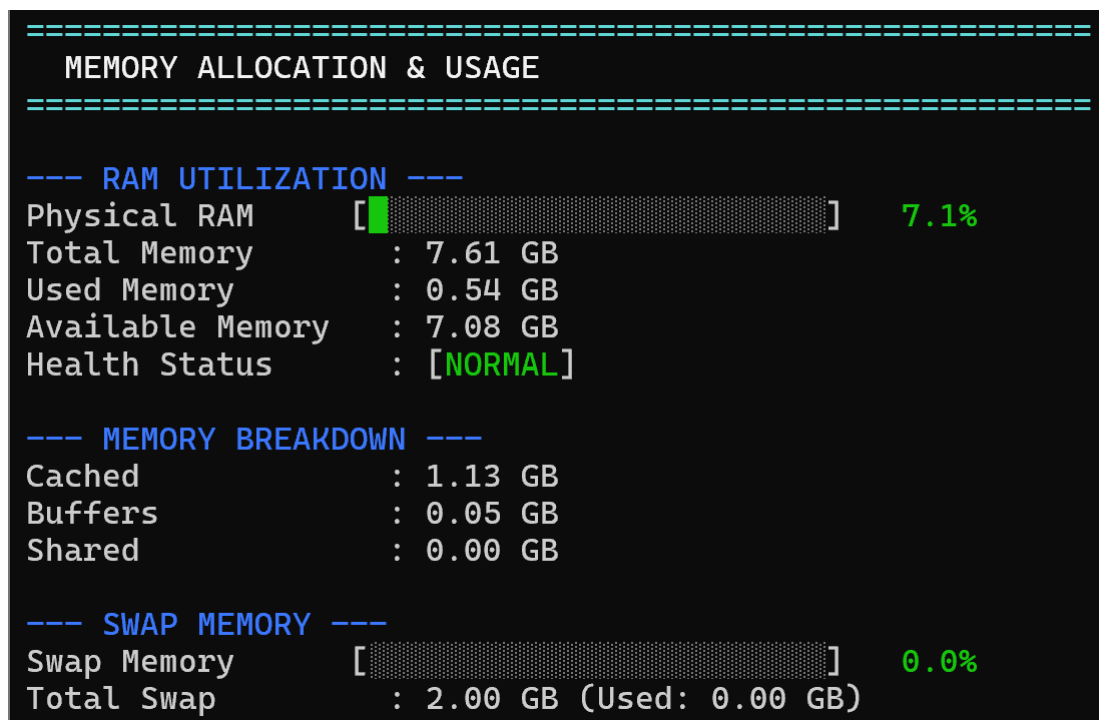


Figure 7.4: Memory (RAM and swap) usage

Option 3 shows the physical RAM bar with total, used and available memory in gigabytes and a status badge, the memory breakdown (cached, buffers, shared), and the swap section with total and used swap, or a message if no swap is configured. All values come from `/proc/meminfo`.

7.7 Disk Monitoring

```
=====
STORAGE & FILESYSTEM USAGE
=====

--- MOUNTED PARTITIONS ---
```

DEVICE	MOUNT	FSTYPE	TOTAL	FREE	USAGE
/dev/sdd	/	ext4	1006.85GB	953.09GB	5.3%
/	[██]		5.3%		
/dev/sdd	/mnt/wslg/distro	ext4	1006.85GB	953.09GB	5.3%
/mnt/wslg/distro	[██]		5.3%		

Figure 7.5: Disk and mounted file-system usage

Option 4 lists the mounted partitions found in /proc/mounts with device, mount point, file-system type, total size, free space and usage percentage, each followed by a progress bar. The space values are calculated from statvfs(). Inode information is not shown in the current version.

7.8 Process Monitoring

```
=====
RUNNING PROCESSES & MONITORING
=====
Process Counts: Total: 25 | Running: 1 | Sleeping: 24 | Stopped: 0 | Zombie: 0

PID      PPID     STATE    COMMAND
-----
1         0        S        systemd
2         1        S        init-systemd(Ub
6         2        S        init
46        1        S        systemd-journal
78        1        S        systemd-resolve
85        1        S        systemd-udevd
115       1        S        chronyd-starter
120       1        S        cron
125       1        S        dbus-daemon
151       1        S        networkd-dispat
158       1        S        systemd-logind
161       1        S        wsl-pro-service
181       1        S        rsyslogd
225       115      S        chronyd
234       225      S        chronyd
272       1        S        agetty
274       1        S        unattended-upgr
333       2        S        SessionLeader
334       333      S        Relay(335)
335       334      S        bash
336       2        S        login
381       1        S        systemd
385       381      S        (sd-pam)
413       336      S        bash
525       335      R        monitor

Submenu Options:
1. View Hierarchical Process Tree
2. Open Process Manager (Kill / Pause / Renice)
3. Return to Main Menu
```

Figure 7.6: Running processes display with process counts and submenu

Figure 7.6 shows the Running Processes screen. The line at the top gives the process counts: 25 processes in total, of which 1 is running, 24 are sleeping, none are stopped and none are zombies. The table lists the PID, the parent PID (PPID), the state and the command name of each process.

PID 1 (system) has PPID 0 and is the root of the process tree; kernel and system services such as system-journal, cron and dbus-daemon have PPID 1. The last entry is the monitor program itself (PID 525, parent bash with PID 335), shown in state R in green, while all other processes are in state S. Below the table, the submenu offers the process tree (option 1), the process manager (option 2) and return to the main menu (option 3). The parent-child relationships of the listed processes are illustrated in Figure 6.5.

7.9 System Uptime

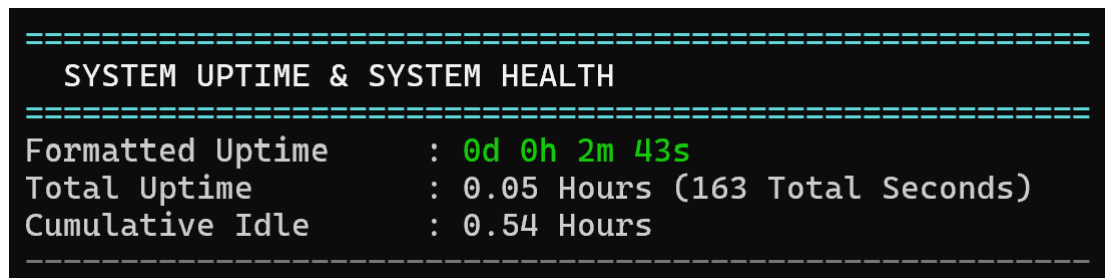


Figure 7.7: System uptime display

Option 6 reads `/proc/uptime` and displays the time since boot as days, hours, minutes and seconds, the total uptime in hours and seconds, and the cumulative idle time in hours.

7.10 System Health

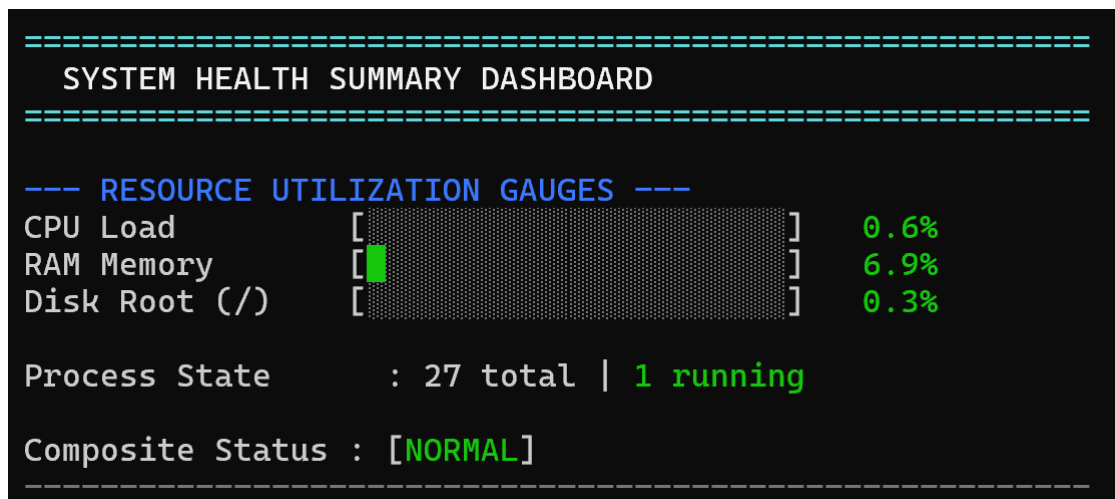


Figure 7.8: System health summary dashboard

Option 7 displays gauges for CPU load, RAM and the root disk, the number of total and running processes, and a composite status that is based on the highest of the three percentages.

7.11 Network Monitoring

```
=====
NETWORK INTERFACE MONITOR
=====
INTERFACE          RECEIVED (RX)      TRANSMITTED (TX)
-----
lo                 3.78 KB           3.78 KB
eth0               7.35 KB           9.29 KB
-----

Options:
 0. Start Real-Time Bandwidth Monitor (KB/s live)
 1. Return to Main Menu
Enter choice: 0

Live bandwidth active. Press 'q', ESC, or End to stop.

=====
REAL-TIME NETWORK SPEED MONITOR (1s refresh)
=====
INTERFACE          DOWNLOAD SPEED      UPLOAD SPEED
-----
lo                 0.00 KB/s         0.00 KB/s
eth0               0.04 KB/s         0.04 KB/s
-----

Press 'q' or ESC to return.

=====
REAL-TIME NETWORK SPEED MONITOR (1s refresh)
=====
INTERFACE          DOWNLOAD SPEED      UPLOAD SPEED
-----
lo                 0.00 KB/s         0.00 KB/s
eth0               0.04 KB/s         0.04 KB/s
-----

Press 'q' or ESC to return.
```

Figure 7.9: Network interface monitoring

Option 8 shows the received and transmitted amount of data of each network interface from `/proc/net/dev`. When the live bandwidth monitor is selected, the screen is refreshed about once per second and shows the download and upload speed of each interface in KB/s until the user presses `q`, `ESC` or `End`.

7.12 Top Resource Processes

```
=====
TOP RESOURCE-CONSUMING PROCESSES
=====

--- TOP 10 BY CPU ---
PID      COMMAND                CPU %
-----
1        systemd                    0.00%
2        init-systemd(Ub             0.00%
6        init                        0.00%
46       systemd-journal             0.00%
78       systemd-resolve              0.00%
84       systemd-udev                 0.00%
143      chronyd-starter              0.00%
144      cron                        0.00%
145      dbus-daemon                  0.00%
149      networkd-dispat              0.00%

--- TOP 10 BY MEMORY (RSS) ---
PID      COMMAND                MEMORY (MB)
-----
234      unattended-upgr            31.93 MB
149      networkd-dispat            29.20 MB
46       systemd-journal            16.33 MB
171      wsl-pro-service            15.05 MB
1        systemd                    15.02 MB
78       systemd-resolve            14.09 MB
364      systemd                    13.05 MB
84       systemd-udev                12.05 MB
162      systemd-logind              9.17 MB
236      chronyd                     6.16 MB
```

Figure 7.10: Top resource-consuming processes

Option 9 samples the process CPU times for 300 ms and then displays two tables: the top ten processes by CPU usage and the top ten by resident memory (in MB).

7.13 Process Search

```
=====
PROCESS SEARCH
=====
Enter process name or PID: 143

PID      PPID      STATE  PROCESS NAME
-----
143      1         S      chronyd-starter

Found 1 matching processes.

Manage PID 143? (1 = Yes, 0 = No): 1

=====
PROCESS MANAGER (SIGNALS & PRIORITY)
=====
Enter Target Process PID: 143

Actions for PID 143:
1. Graceful Terminate (SIGTERM - 15)
2. Force Kill (SIGKILL - 9)
3. Pause Process (SIGSTOP - 19)
4. Resume Process (SIGCONT - 18)
5. Change Priority (renice)
6. Cancel
Select action (1-6): |
```

Figure 7.11: Process search

Option 10 asks for a process name or PID. The result table lists the matching processes with PID, PPID, state and name, followed by the number of matches. If a single process is found, the user may open the process manager for it.

7.14 Resource Alerts

```
=====
RESOURCE THRESHOLD ALERTS
=====

--- ACTIVE TELEMETRY & WARNING CHECKS ---
CPU Usage:    0.0% [NORMAL]
RAM Usage:    7.2% [NORMAL]
Disk (/):     0.3% [NORMAL]

All system metrics are operating within safe parameters.
```

Figure 7.12: Resource threshold alerts

Option 14 shows the CPU, RAM and root-disk usage with their status badges and prints a notice above 70 % or an alert above 85 %. If all values are below the thresholds, a message states that all metrics are within safe parameters.

7.15 Resource Logging

```
=====
RESOURCE LOGGING FACILITY
=====

Logged system snapshot to 'monitor_log.txt'.
Timestamp : 2026-10-04 06:23:07
Recorded  : CPU    0.0% | RAM    7.2% | DISK    0.3%
```

Figure 7.13: Resource logging facility

Figure 7.13 shows the Resource Logging screen. The program reports that a system snapshot was logged to 'monitor_log.txt' and displays the timestamp (2026-10-04 06:23:07) and the recorded values (CPU 0.0 %, RAM 7.2 %, DISK 0.3 %). These values describe the state of the system at the moment of logging. Each time the option is used, one more line is appended to the file.

7.16 Interactive Process Manager

```
Submenu Options:
  1. View Hierarchical Process Tree
  2. Open Process Manager (Kill / Pause / Renice)
  3. Return to Main Menu
Enter choice: 2

=====
  PROCESS MANAGER (SIGNALS & PRIORITY)
=====
Enter Target Process PID: 234

Actions for PID 234:
  1. Graceful Terminate (SIGTERM - 15)
  2. Force Kill          (SIGKILL - 9)
  3. Pause Process      (SIGSTOP - 19)
  4. Resume Process     (SIGCONT - 18)
  5. Change Priority    (renice)
  6. Cancel
```

Figure 7.15: Process manager showing the action menu for the entered PID

Figure 7.15 shows the process manager opened from the process submenu. The user entered PID 234, and the program displays the six actions available for it: graceful terminate (SIGTERM, 15), force kill (SIGKILL, 9), pause (SIGSTOP, 19), resume (SIGCONT, 18), change priority (renice) and cancel. The screenshot shows the menu only; the result of an action is not shown.

7.17 System Load Averages and Context Switches

```
=====
SYSTEM LOAD AVERAGE & KERNEL SCHEDULING
=====

--- CPU LOAD AVERAGES (Multi-Core Normalized) ---
1-Minute Load      : 0.03 (0.2% per core)
1m Load Gauge     [████████████████████████████████████████] 0.2%
5-Minute Load      : 0.03 (0.2% per core)
5m Load Gauge     [████████████████████████████████████████] 0.2%
15-Minute Load     : 0.00 (0.0% per core)
15m Load Gauge     [████████████████████████████████████████] 0.0%

--- SCHEDULER & THREAD POOL METRICS ---
Runnable Threads    : 2 active / 242 total
Last Spawned PID    : 564
Configured CPU Cores : 12 online

--- CONTEXT SWITCHES & PROCESS CONCURRENCY ---
Total Context Switches : 130394
Total Forks Since Boot : 816
Processes In Run Queue : 4
Processes Blocked      : 0 (waiting for I/O completion)
```

Figure 7.16: System load averages and context switches

Option 12 displays the 1, 5 and 15 minute load averages and their value per core, the runnable and total scheduling entities, the last assigned PID, the number of online cores, and the counters for context switches, forks since boot, processes in the run queue and blocked processes.

7.18 Kernel Modules Explorer

```
=====
LINUX KERNEL MODULES (LKM) & DRIVERS
=====
Kernel Subsystem Summary: 15 Loaded Modules | Memory: 1.88 MB

MODULE NAME                SIZE (KB)    USED BY    STATE
-----
intel_rapl_msr             16.0 KB      0         Live
intel_rapl_common          40.0 KB      1         Live
kvm_intel                  360.0 KB     0         Live
kvm                        980.0 KB     1         Live
irqbypass                 16.0 KB      1         Live
battery                   20.0 KB      0         Live
ac                         16.0 KB      0         Live
sch_fq_codel              16.0 KB      1         Live
autofs4                   44.0 KB      0         Live
br_netfilter              28.0 KB      0         Live
bridge                    280.0 KB     1         Live
stp                       12.0 KB      1         Live
llc                       12.0 KB      2         Live
ip_tables                 28.0 KB      0         Live
tun                       52.0 KB      0         Live

Options:
  1. Search for a specific kernel module
  2. Continue
Enter choice: 1
Enter module name substring: ac

--- MATCHING KERNEL MODULES ---
MODULE NAME                SIZE (KB)    USED BY    STATE
-----
ac                         16.0 KB      0         Live
-----
```

Figure 7.17: Kernel modules explorer

Option 13 displays the number of loaded kernel modules, their total memory and a table of the first 20 modules with size, use count and state. A name substring can then be entered to search among all modules. The content of `/proc/modules` can differ between a native Linux system and WSL.

7.19 Testing

The program was tested by running each menu option and checking that the expected screen appears. Table 7.1 lists the test cases. In the column “Actual Result”, only the cases that are

supported by the screenshots in this report are marked as verified. The remaining cases should be completed by the team after executing them; no results have been assumed.

Table 7.1: Test cases and execution results

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC01	Start application	./monitor	Main menu with 16 options is displayed	Verified during execution (Figure 7.1)	Pass
TC02	Valid menu selection	1	Selected module is executed and the Action prompt is shown	Verified during execution (Figures 7.2, 7.6)	Pass
TC03	Invalid menu selection	99 / non-numeric text	Error message is printed and the menu is displayed again	[Record after execution]	[Pass / Fail]
TC04	System information	1	OS, kernel, architecture, host name, CPU model, cores, frequency and user are shown	Successfully displays the corresponding system information (Figure 7.2)	Pass
TC05	CPU monitoring	2, then 0	CPU bar with status badge; live line updates until q	[Record after execution]	[Pass / Fail]
TC06	Memory monitoring	3	RAM, breakdown and swap information is shown	[Record after execution]	[Pass / Fail]
TC07	Disk monitoring	4	Table of mounted partitions with usage bars	[Record after execution]	[Pass / Fail]
TC08	Process monitoring	5	Process counts, process table (first 30) and submenu are shown	Verified during execution (Figure 7.6)	Pass
TC09	System uptime	6	Uptime in d/h/m/s, total hours and idle hours	[Record after execution]	[Pass / Fail]
TC10	Health summary	7	Gauges, process counts and composite status	[Record after execution]	[Pass / Fail]
TC11	Network monitoring	8, then 0	Interface table; live KB/s table until q	[Record after execution]	[Pass / Fail]
TC12	Top processes	9	Top 10 by CPU and top 10 by memory	[Record after execution]	[Pass / Fail]
TC13	Process search	10, name or PID	Matching processes or “No matching processes” message	[Record after execution]	[Pass / Fail]
TC14	Resource alerts	14	Badges and notice/alert messages as per thresholds	[Record after execution]	[Pass / Fail]
TC15	Resource logging	15	A line is appended to monitor_log.txt and a confirmation is shown	Verified during execution (Figure 7.13)	Pass
TC16	Exit operation	16	Exit message is printed and the program ends	[Record after execution]	[Pass / Fail]
TC17	Process manager	11 (or 5, then 2), PID	Action menu is shown for an existing PID; invalid or missing PID is rejected	Action menu displayed (Figure 7.15); action results not yet recorded	Partial
TC18	Load averages	12	Load averages, run queue and context-switch counters	[Record after execution]	[Pass / Fail]
TC19	Kernel modules	13	Module summary, table and search	[Record after execution]	[Pass / Fail]

7.20 Demonstration of OS Concepts

Table 7.2 summarises which operating system concept each feature demonstrates and how it is implemented.

Table 7.2: Demonstration of OS concepts

Feature	OS Concept	Implementation
System information	OS services / system calls	uname(), sysconf(), getpwuid(), /proc/cpuinfo, /etc/os-release
CPU monitoring	CPU / resource management	Difference of /proc/stat time counters over a 300 ms interval
Memory monitoring	Memory management, virtual memory (swap)	/proc/meminfo: MemTotal, MemAvailable, cache, buffers, SwapTotal, SwapFree
Disk monitoring	File system management	/proc/mounts and statvfs() block statistics
Process monitoring	Process management, process states	/proc/[PID]/stat: PID, PPID, state; counts by state
Process tree	Process hierarchy	Recursive search of children by PPID, starting from PPID 0
Top processes	Process resource analysis	utime + stime from /proc/[PID]/stat, RSS from /proc/[PID]/statm, qsort()
Process manager	Process control, signals, scheduling priority	kill() with SIGTERM/SIGKILL/SIGSTOP/SIGCONT; setpriority()
Load averages and context switches	CPU scheduling	/proc/loadavg and ctxt, procs_running, procs_blocked from /proc/stat
Kernel modules	Kernel / system interface	/proc/modules
Network monitoring	Resource management (I/O)	/proc/net/dev byte counters and their differences
/proc usage	Kernel / system interface	Kernel data read as ordinary files
Health summary and alerts	Resource management	Combination of CPU, memory and disk values and comparison with thresholds
Logging	File handling	fopen() in append mode, fprintf(), time functions
Live monitors	Terminal I/O and signals	termios raw mode, select(), SIGINT/SIGTERM handler

CHAPTER 8

CONCLUSION

8.1 Summary of the Project

The Linux System Information and Resource Monitoring Tool is a menu-driven terminal application written in C for Linux/Ubuntu. It reads information directly from the /proc filesystem and from Linux system calls and library functions and shows it through a single interface. The application provides system information, CPU usage with a live view, RAM and swap usage, disk usage of mounted file systems, a list of running processes with a process tree, system uptime, a health summary, network counters with a live bandwidth view, the top CPU and memory processes, process search, a process manager for signals and priority, load averages and context-switch counters, a kernel modules explorer, resource alerts and resource logging.

During the project, the team applied concepts from the Operating Systems course to a real program: system calls, process states and hierarchy, memory accounting, file-system statistics and resource management. The program is intended as an educational tool and not as a replacement for professional monitoring software.

8.2 Objectives Achieved

Table 8.1: Objectives and implemented results

No.	Objective (Section 1.4)	Implemented result
1	Develop a Linux monitoring application in C	main.c compiled with GCC into the executable monitor
2	Collect system information through Linux interfaces	uname(), sysconf(), statvfs(), /proc files (Table 4.2)
3	Monitor CPU utilisation	CPU module with 300 ms sampling and live mode (Section 6.1.2)
4	Monitor RAM and swap	Memory module (Section 6.1.3)
5	Monitor disk space usage	Disk module for mounted file systems (Section 6.1.4); inode usage not included
6	Inspect processes and hierarchy	Process module with counts, list and tree (Section 6.1.5)
7	Monitor network activity	Network module with cumulative counters and live KB/s (Section 6.1.8)
8	Identify resource-consuming processes	Top 10 by CPU and by memory (Section 6.1.9)
9	Search processes; send signals and change priority	Process search and process manager (Sections 6.1.10, 6.1.13)
10	Provide alerts and logging	Alert module and monitor_log.txt (Sections 6.1.11, 6.1.12)
11	Understand OS concepts through a program	Concept mapping in Tables 2.3 and 7.2

8.3 Results Achieved

The application starts from a single command and gives access to all the modules through one menu. The modules that were executed and captured in this report (main menu, system information, process monitoring, process manager menu and resource logging) display the expected information. The program reads data directly from the kernel interfaces, is lightweight, and records snapshots in a log file that can be inspected later. Every value shown on the screen can be traced to a specific `/proc` file or system call in the source code. No numerical performance results are claimed in this report; the values shown in the screenshots only describe the state of the test system at the time they were captured.

8.4 Skills and Learning Outcomes

- **C programming:** structures, pointers, string parsing with `sscanf()`, dynamic memory, function organisation and sorting with `qsort()`.
- **Linux system programming:** use of system calls and library functions for system information, file systems, signals and priorities.
- **/proc filesystem:** understanding the format and meaning of `/proc/stat`, `/proc/meminfo`, `/proc/uptime`, `/proc/net/dev` and `/proc/[PID]/` files.
- **Process management:** process states, PID and PPID relationships, and signals.
- **Memory management:** the difference between total, free and available memory, caches, swap and resident memory.
- **File systems:** mounted file systems, block statistics and file handling for the log file.
- **Resource monitoring:** calculating utilisation from cumulative counters and sampling intervals.
- **Debugging:** finding and correcting parsing and boundary errors using compiler warnings and test runs.
- **GCC compilation:** using warning options and building a program from the command line.
- **Linux terminal usage:** navigation, compilation, execution, and working under both Ubuntu and WSL.
- **Teamwork:** dividing the modules among team members and integrating them in one program, with a shared GitHub repository.

8.5 Limitations

- The interface is terminal-based; there is no graphical dashboard or real-time graph.

- The implementation is specific to Linux because it depends on /proc and Linux system interfaces.
- There is no remote monitoring and no database; the only stored data is the text log file.
- IPC mechanisms (pipes, shared memory, message queues, semaphores) are not directly implemented, and the program is single-threaded.
- Inode statistics are not displayed, and the Active and Inactive memory values are read but not shown.
- CPU usage is shown for the whole system only, not per logical core. The live view is sampled about every 0.7 seconds, not continuously.
- The CPU module and menu refer to a live frequency monitor, but frequency is shown only once, in the System Information screen; the live view shows CPU utilisation.
- The uptime module shows cumulative idle time, not total busy CPU time.
- Fixed limits apply: 4096 processes, 32 mounts, 16 network interfaces, and the first 20 kernel modules; the process list shows the first 30 processes and the tree is limited to four levels.
- Network speeds are calculated for an interval that is assumed to be one second, so they are approximate.
- Logging is manual: a snapshot is written only when the user selects the option. The thresholds (70 % and 85 %) are fixed in the source code, and alerts cover CPU, RAM and the root disk only.
- The process manager does not ask for confirmation before sending a signal, and its operations depend on user permissions.
- The values reported depend on the Linux environment. Under WSL, the processes, resources and kernel information are those of the WSL virtual machine and can differ from native Linux.

8.6 Future Scope

The following improvements are possible in future versions. They are not features of the current application.

- A graphical user interface.
- Real-time graphs for CPU, memory and network usage.
- Advanced network statistics, such as packets, errors and per-connection information.
- Inode usage display and per-core CPU usage.

- Temperature monitoring.
- Remote monitoring of other machines.
- Historical analytics based on the log file, and automatic periodic logging.
- Report generation, for example export to a file.
- Additional process management features, such as confirmation prompts and sorting options.
- Notifications when thresholds are crossed.
- Configurable thresholds instead of fixed values.

CHAPTER 9

REFERENCES

9.1 Books

- [1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.
- [3] M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.
- [4] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed., Addison-Wesley, 2013.
- [5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988.
- [6] R. Love, *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010.

9.2 Research Papers

No research papers were used directly in the implementation of this project. [ADD ANY PAPERS THAT THE TEAM ACTUALLY READ AND CITED, OR LEAVE THIS STATEMENT.]

9.3 Linux / GNU Documentation

The following documentation was used for the interfaces implemented in main.c. Last accessed: [ACCESS DATE].

- [7] Linux man-pages project, `uname(2)`. <https://man7.org/linux/man-pages/man2/uname.2.html>
- [8] Linux man-pages project, `sysconf(3)`. <https://man7.org/linux/man-pages/man3/sysconf.3.html>
- [9] Linux man-pages project, `statvfs(3)`. <https://man7.org/linux/man-pages/man3/statvfs.3.html>
- [10] Linux man-pages project, `proc(5)` – process information pseudo-filesystem. <https://man7.org/linux/man-pages/man5/proc.5.html>
- [11] Linux man-pages project, `proc_pid_stat(5)` – `/proc/pid/stat`. https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html
- [12] Linux man-pages project, `proc_pid_statm(5)` – `/proc/pid/statm`. https://man7.org/linux/man-pages/man5/proc_pid_statm.5.html
- [13] Linux man-pages project, `proc_meminfo(5)` – `/proc/meminfo`. https://man7.org/linux/man-pages/man5/proc_meminfo.5.html

- [14] Linux man-pages project, `proc_stat(5)` – `/proc/stat`. https://man7.org/linux/man-pages/man5/proc_stat.5.html
- [15] Linux man-pages project, `proc_uptime(5)` – `/proc/uptime`. https://man7.org/linux/man-pages/man5/proc_uptime.5.html
- [16] Linux man-pages project, `proc_loadavg(5)` – `/proc/loadavg`. https://man7.org/linux/man-pages/man5/proc_loadavg.5.html
- [17] Linux man-pages project, `proc_pid_net(5)` – `/proc/net/` (including `/proc/net/dev`); also available as `proc_net(5)`. https://man7.org/linux/man-pages/man5/proc_pid_net.5.html
- [18] Linux man-pages project, `proc_modules(5)` – `/proc/modules`. https://man7.org/linux/man-pages/man5/proc_modules.5.html
- [19] Linux man-pages project, `kill(2)`, `setpriority(2)`, `select(2)`, `signal(7)`, `termios(3)`, `qsort(3)`, `strcasestr(3)`. <https://man7.org/linux/man-pages/>
- [20] GCC, the GNU Compiler Collection – Online Documentation. <https://gcc.gnu.org/onlinedocs/>
- [21] The GNU C Library Reference Manual. <https://www.gnu.org/software/libc/manual/>

9.4 Online Resources

- [22] The Linux kernel documentation, The `/proc` Filesystem. <https://docs.kernel.org/filesystems/proc.html>
- [23] The Open Group Base Specifications Issue 7 (POSIX.1-2008). <https://pubs.opengroup.org/onlinepubs/9699919799/>
- [24] Linux man-pages project, online manual pages. <https://man7.org/linux/man-pages/>
- [25] Project repository: V-Max OSSP Project. https://github.com/V-max-OSSP/V-Max_OSSP_Project